

UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL-REI

Departamento de Engenharia de Telecomunicações

Thierry Gonçalves Braga

**PIPELINE MODULAR PARA DETECÇÃO DE
ÁUDIO SINTÉTICO
USANDO ARQUITETURAS NEURAIIS HÍBRIDAS
E
ANÁLISE EMPÍRICA DE CARACTERÍSTICAS**

São João del-Rei — MG

2026

Thierry Gonçalves Braga

**PIPELINE MODULAR PARA DETECÇÃO DE
ÁUDIO SINTÉTICO
USANDO ARQUITETURAS NEURAIS HÍBRIDAS
E
ANÁLISE EMPÍRICA DE CARACTERÍSTICAS**

Trabalho de Conclusão de Curso apresentado ao Departamento de Engenharia de Telecomunicações da Universidade Federal de São João del-Rei, como requisito parcial para a obtenção do título de Bacharel em Engenharia de Telecomunicações.

Orientadora: Prof.^a Ana Cláudia

FOLHA DE APROVAÇÃO

Trabalho de Conclusão de Curso defendido e aprovado em _____, como requisito parcial para obtenção do título de Bacharel em Engenharia de Telecomunicações.

Banca Examinadora:

Prof.^a Ana Cláudia (Orientadora)
Universidade Federal de São João del-Rei

Prof. _____
Instituição: _____

Prof. _____
Instituição: _____

RESUMO

Este trabalho apresenta o desenvolvimento e a avaliação de um *pipeline* modular e de código aberto para detecção de áudio sintético, em resposta à proliferação de *deepfakes* de voz. A metodologia integra pré-processamento acústico (detecção de atividade de voz e controle automático de ganho), extração de características heterogêneas — áudio bruto, representações tempo-frequência (espectrogramas Mel) e um vetor tabular de descritores temporais, cepstrais (MFCC) e espectro-perceptuais (RASTA-PLP) —, treinamento supervisionado, inferência padronizada e geração automática de relatórios de *benchmark*. A versão experimental consolidada avalia 11 modelos reconhecidos na literatura, abrangendo modelos auto-supervisionados (SSL), redes de áudio bruto, grafos de atenção, *Transformers* espectrais, redes convolucionais multi-escala e classificadores estatísticos clássicos. Os experimentos utilizaram um *corpus* balanceado de 15.000 amostras de fala em português, com partição estratificada 70/15/15 e janelas monofônicas padronizadas em 16 kHz e 5 s. Os modelos neurais foram treinados em perfil GPU (WSL2/CUDA), com métricas de inferência extraídas no ambiente de execução de cada modelo (CPU ou GPU, conforme a família); SVM e *Random Forest* foram otimizados por validação cruzada. No conjunto de teste limpo, HuBERT e Conformer obtiveram 99,87% e 99,82% de acurácia, seguidos por Res2Net e WavLM (ambos 99,69%), SVM (99,24%) e CCT (99,20%). Sob ruído aditivo a 10 dB de SNR, WavLM, Conformer, Res2Net e HuBERT preservaram mais de 96% de acurácia, enquanto AASIST e RawGAT-ST degradaram para 88,7% e 84,0% — evidenciando que acurácia em distribuição fechada é indicador insuficiente de qualidade e que a expressividade da representação de entrada e a capacidade do modelo são fatores de primeira ordem para a robustez. Os resultados confirmam a funcionalidade do *pipeline* para comparação padronizada de arquiteturas, mas também evidenciam, conforme a análise de vazamento de domínio documentada, a necessidade de validação externa, robustez a degradações reais de canal (codecs, reverberação, ruído não estacionário) e controle de vieses de fonte antes do uso em cenários reais.

Palavras-chave: *deepfake* de áudio; *anti-spoofing*; arquiteturas neurais; extração de características; *min t-DCF*; robustez a ruído.

ABSTRACT

This work presents the development and evaluation of a modular open-source pipeline for synthetic audio detection, addressing the proliferation of voice deepfakes. The methodology integrates acoustic preprocessing (voice activity detection and automatic gain control), extraction of heterogeneous features — raw audio, time-frequency representations (Mel spectrograms), and a tabular vector of temporal, cepstral (MFCC), and spectro-perceptual (RASTA-PLP) descriptors —, supervised training, standardized inference, and automatic benchmark report generation. The consolidated experimental version evaluates 11 models recognized in the literature, spanning self-supervised (SSL) models, raw-audio networks, graph-attention networks, spectral Transformers, multi-scale convolutional networks, and classical statistical classifiers. Experiments used a balanced 15,000-sample Portuguese speech corpus, with a stratified 70/15/15 split and 16 kHz mono 5 s

windows. Neural models were trained on a GPU profile (WSL2/CUDA), with inference metrics extracted in each model’s execution environment (CPU or GPU, depending on the family); SVM and Random Forest were optimized through cross-validation. On the clean test set, Res2Net and Conformer achieved 99.69% accuracy, followed by Audio Spectrogram Transformer (98.71%), Random Forest (98.18%), and RawNet2 (97.38%). Under additive noise at 10 dB SNR, Conformer, AST, and Res2Net preserved over 96% accuracy while the classical classifiers degraded to 66–68% — showing that closed-distribution accuracy is an insufficient quality indicator and that input-representation expressiveness and model capacity are first-order factors for robustness. The results confirm the pipeline’s ability to compare heterogeneous architectures in a standardized way, while also showing, consistent with the documented domain-leakage analysis, the need for external validation, robustness to real channel degradations (codecs, reverberation, non-stationary noise), and source bias control before real-world deployment.

Keywords: audio deepfake; anti-spoofing; neural architectures; feature extraction; min t-DCF; noise robustness.

Lista de Figuras

1	Diagrama do <i>pipeline</i> proposto. O modelo treinado (b) é implantado (<i>deploy</i>) na fase de inferência (a). A pontuação p_{fake} é a probabilidade predita de a amostra ser sintética: valores acima do limiar de decisão θ resultam em SINTÉTICO. O limiar é 0,5 por padrão; para WavLM Original e HuBERT Original, é calibrado em validação com ruído ($\theta \approx 0,48$; Seção 4.5).	24
2	Acurácia e AUC-ROC no conjunto de teste limpo.	40
3	Robustez a AWGN na forma de onda (acurácia vs. SNR).	41
4	Taxa de Erro Igual (EER) por arquitetura.	42
5	t -DCF* por arquitetura.	43
6	Latência e tamanho dos modelos.	53
7	Trade-off entre acurácia, latência e tamanho dos artefatos.	54
8	Resumo descritivo das curvas de validação em execução única; a diferença pico-final não demonstra, isoladamente, estabilidade de otimização. Modelos clássicos não possuem trajetória por época.	54
9	Matrizes de confusão dos onze modelos avaliados no artigo.	55
10	Importância de Gini das 15 características mais relevantes do <i>Random Forest</i> , coloridas por família (temporal, MFCC, RASTA-PLP). Agregadas, as famílias respondem por 14,9%, 49,9% e 35,2% da importância total, respectivamente.	55
11	Distribuições da pontuação por classe para AASIST e RawGAT-ST	56
12	Curvas DET dos onze modelos no conjunto de teste limpo	56

Lista de Tabelas

1	Principais modalidades de geração e manipulação de fala sintética.	17
2	<i>Survey</i> das gerações de métodos de detecção de áudio sintético.	17
3	Representação de entrada consumida por família de modelo.	27
4	Descritores acústicos disponíveis no <i>pipeline</i> : síntese da literatura. Custo Computacional : estimativa qualitativa (Baixo/Médio/Alto) do consumo de tempo e memória para extração.	30
5	Composição analítica da base de dados (15.000 amostras) estratificada por classe e proveniência estrutural. A disjunção de locutores entre as partições de treino/validação/teste não é garantida (Seção 4.4.3).	31
6	Especificações técnicas do protocolo de pré-processamento acústico e partições de amostragem.	31
7	Especificações de infraestrutura, hardware e parametrização do ambiente de <i>benchmarking</i>	36
8	Resultados consolidados no conjunto de teste limpo.	37
9	Eficiência computacional dos modelos consolidados.	37
10	Robustez a ruído AWGN, aplicada à forma de onda antes dos frontends, por modelo consolidado.	38
11	Resumo descritivo das curvas de validação em execução única.	39
12	Ambiente de software e parâmetros de reprodução das execuções.	57
13	Hiperparâmetros efetivos por modelo	57
14	Módulos reais do <i>pipeline</i> referenciados pelo <i>benchmark</i>	58

Lista de Abreviaturas e Siglas

AGC	<i>Automatic Gain Control</i> (Controle Automático de Ganho)
AASIST	<i>Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks</i>
AM-Softmax	<i>Additive Margin Softmax</i>
AST	<i>Audio Spectrogram Transformer</i>
AUC-ROC	<i>Area Under the Receiver Operating Characteristic Curve</i>
AWGN	<i>Additive White Gaussian Noise</i> (Ruído Gaussiano Branco Aditivo)
CCT	<i>Compact Convolutional Transformer</i>
CNN	<i>Convolutional Neural Network</i> (Rede Neural Convolucional)
CPU	<i>Central Processing Unit</i>
CQT	<i>Constant-Q Transform</i>
CQCC	<i>Constant-Q Cepstral Coefficients</i>
DET	<i>Detection Error Tradeoff</i>
EER	<i>Equal Error Rate</i> (Taxa de Erro Igual)
F0	Frequência Fundamental
FFT	<i>Fast Fourier Transform</i> (Transformada Rápida de Fourier)
FMS	<i>Feature Map Scaling</i> (Escalação de Mapas de Características)
GAT	<i>Graph Attention Network</i>
GMM-UBM	<i>Gaussian Mixture Model – Universal Background Model</i>
GPU	<i>Graphics Processing Unit</i>
GRU	<i>Gated Recurrent Unit</i>
HNR	<i>Harmonic-to-Noise Ratio</i> (Relação Harmônico-Ruído)
HS-GAL	<i>Heterogeneous Stacking Graph Attention Layer</i>

LCNN	<i>Light CNN</i>
LFCC	<i>Linear Frequency Cepstral Coefficients</i>
LSTM	<i>Long Short-Term Memory</i>
LUFS	<i>Loudness Units relative to Full Scale</i>
MFCC	<i>Mel-Frequency Cepstral Coefficients</i> (Coeficientes Cepstrais na Escala Mel)
MLP	<i>Multilayer Perceptron</i>
MLS	<i>Multilingual LibriSpeech</i>
PCM	<i>Pulse-Code Modulation</i>
RBF	<i>Radial Basis Function</i> (Função de Base Radial)
RNN	<i>Recurrent Neural Network</i>
RASTA-PLP	<i>RelAtive SpecTrA – Perceptual Linear Prediction</i>
RVC	<i>Retrieval-based Voice Conversion</i>
SHAP	<i>SHapley Additive exPlanations</i>
SNR	<i>Signal-to-Noise Ratio</i> (Relação Sinal-Ruído)
SSL	<i>Self-Supervised Learning</i> (Aprendizado Auto-Supervisionado)
STFT	<i>Short-Time Fourier Transform</i>
SVM	<i>Support Vector Machine</i> (Máquina de Vetores de Suporte)
t-DCF	<i>Tandem Detection Cost Function</i>
TTS	<i>Text-to-Speech</i> (Síntese de Fala)
VAD	<i>Voice Activity Detection</i> (Detecção de Atividade de Voz)
VC	<i>Voice Conversion</i> (Conversão de Voz)
ViT	<i>Vision Transformer</i>
XTTS	<i>Cross-lingual Text-to-Speech</i>
ZCR	<i>Zero-Crossing Rate</i> (Taxa de Cruzamento por Zero)

Sumário

1	Introdução	11
1.1	Biometria de Voz: Contexto de Aplicação	12
1.2	Objetivos	13
1.2.1	Objetivo Geral	13
1.2.2	Objetivos Específicos	13
1.3	Questões de Pesquisa	13
1.4	Organização do Trabalho	14
2	Tecnologias de Síntese de Voz	15
2.1	Da Síntese Clássica à Síntese Neural	15
2.2	Text-to-Speech, Clonagem e Conversão de Voz	15
2.3	Implicações para a Detecção	17
2.4	Posicionamento em Relação à Literatura	18
3	Arquiteturas de Classificação	19
3.1	Modelos Auto-Supervisionados (SSL) e de Áudio Bruto	19
3.2	Arquiteturas Neurais de Domínio Espectro-Temporal	21
3.3	Classificadores Estatísticos Clássicos	22
4	Metodologia	24
4.1	Visão Geral do Pipeline	24
4.2	Pré-processamento: Normalização e Detecção de Atividade de Voz	25
4.3	Representações de Entrada e Características Acústicas	27
4.4	Conjunto de Dados e Protocolo Experimental	30
4.4.1	Composição e Proveniência	30
4.4.2	Pré-processamento e Particionamento	31
4.4.3	Limitações do Protocolo: Viés de Canal e Sobreposição de Locutores	32
4.5	Métricas de Avaliação	32
4.6	Ferramentas de Software Livre	34
4.7	Protocolo de Treinamento	35
5	Experimentos e Resultados	36
5.1	Ambiente Experimental e Hardware	36
5.2	Resultados Consolidados	37
5.3	Análise dos Resultados	38

6	Conclusões	45
6.1	Principais Contribuições	46
6.2	Limitações	46
6.3	Trabalhos Futuros	47
A	Configurações Detalhadas do Pipeline	57
A.1	Hiperparâmetros Consolidados	57
A.2	Organização dos Artefatos	58
B	Implementação de Referência	58
C	Comandos de Execução e Reprodutibilidade	58

1 Introdução

A síntese de fala, ou *Text-to-Speech* (TTS), evoluiu de sistemas mecânicos rudimentares para modelos computacionais capazes de reproduzir, com alta fidelidade, as nuances, a emoção e a identidade de um locutor específico — processo hoje conhecido como clonagem de voz [1]. Os primeiros marcos da síntese eletrônica surgiram com os *Vocoders* (*Voice Coders*) de Homer Dudley, na década de 1930 [2]. Baseados no modelo fonte-filtro de produção de fala, esses dispositivos separavam a excitação (a fonte, como as cordas vocais) das características do trato vocal (o filtro). Embora revolucionários e fundamentais para a compressão em telecomunicações, os sistemas puramente paramétricos produziam vozes de aspecto robótico, carentes de naturalidade [3].

Para superar essa limitação, a década de 1990 foi marcada pela síntese concatenativa, que se baseia na gravação de extensos bancos de voz humana segmentados em pequenas unidades fonéticas (difones ou sílabas) [4]. A verdadeira quebra de paradigma, contudo, ocorreu na última década com o advento do aprendizado profundo (*Deep Learning*). Modelos neurais generativos, como o WaveNet [5] e o Tacotron [6], substituíram os *pipelines* tradicionais por arquiteturas de ponta a ponta (*end-to-end*), aprendendo o mapeamento direto entre texto (ou fonemas) e a forma de onda acústica, capturando variações prosódicas complexas. É nesse cenário que a clonagem de voz se torna viável e acessível: por meio de adaptação de locutor e aprendizado *few-shot* ou *zero-shot*, tornou-se possível sintetizar a voz de um indivíduo com poucos segundos de áudio de referência [1, 7].

A crescente democratização dessas tecnologias tem facilitado a criação de *deepfakes* de áudio com qualidade cada vez mais próxima à voz humana natural [8, 9]. Sistemas neurais como Tacotron 2 [10] e WaveNet [5] estabeleceram o patamar de naturalidade da fala sintética, enquanto abordagens de clonagem *few-shot* [1, 7] e sistemas de conversão de voz em tempo real, como o RVC (*Retrieval-based Voice Conversion*), cuja produção já motiva detectores dedicados [11], replicam uma voz-alvo com poucos segundos de áudio de referência. Embora benéfica para aplicações legítimas (assistentes virtuais, dublagem automatizada), essa evolução tem sido explorada maliciosamente em fraudes financeiras, manipulação política e chantagem [12, 13]. No contexto brasileiro, o DFRLab identificou 78 casos confirmados ou alegados de *deepfakes* políticos entre maio e outubro das eleições municipais de 2024 [13], além de documentar os desafios de identificação desse conteúdo no período pré-eleitoral [14] — evidenciando a necessidade de sistemas de detecção robustos voltados ao português brasileiro.

Estudos perceptuais indicam que humanos identificam corretamente *deepfakes* de áudio em cerca de 73% dos casos (e fala genuína em apenas 68%), com ganhos apenas marginais mesmo após familiarização com exemplos [8]. Isso justifica o desenvolvimento de sistemas automatizados e explica o surgimento de *benchmarks* padronizados como o ASVspoof [15–17] e o desafio ADD [18], hoje eixos centrais de avaliação.

1.1 Biometria de Voz: Contexto de Aplicação

A biometria de voz — autenticação por características vocais individuais — é um dos principais contextos de aplicação para detectores de áudio sintético, e sua adoção crescente amplia a superfície de ataque dos *deepfakes* de voz. Relatórios do setor indicam expansão sustentada da adoção — e, simultaneamente, dos ataques com áudio sintético [12] —, puxada por instituições que lidam com alto volume de transações críticas remotas: o setor financeiro (autenticação passiva durante o atendimento), telecomunicações (mitigação de fraudes como o *SIM swapping*), saúde (acesso a prontuários eletrônicos) e o setor público (concessão de benefícios e controle de acesso). Nesses cenários, um sistema de verificação de locutor opera tipicamente em *tandem* com uma contramedida anti-*spoofing* — exatamente a configuração modelada pela métrica *t*-DCF adotada neste trabalho (Seção 4.5) — o que torna a detecção robusta de áudio sintético um requisito operacional, e não apenas acadêmico.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver e avaliar um *pipeline* modular, reproduzível e de código aberto para detecção de áudio sintético em português, contemplando 11 modelos consolidados na literatura e permitindo a comparação sistemática entre representações de áudio bruto, descritores acústicos tabulares e mapas tempo-frequência.

1.2.2 Objetivos Específicos

- Revisar as principais gerações de síntese, clonagem e conversão de voz, relacionando seus artefatos acústicos aos desafios de detecção.
- Consolidar uma base experimental balanceada em português, com particionamento reproduzível e especificação padronizada de taxa de amostragem, duração, canal e rótulo.
- Implementar pré-processamento acústico com detecção de atividade de voz (VAD), controle automático de ganho (AGC), normalização temporal e extração de características compatíveis com cada família de modelo.
- Implementar e comparar 11 modelos referenciados na literatura: WavLM, HuBERT, RawNet2, AASIST, RawGAT-ST, Conformer, CCT, AST, Res2Net, SVM e *Random Forest*.
- Avaliar os modelos sob um protocolo comum, reportando acurácia, F1-score, AUC-ROC, EER, *t*-DCF, latência, tamanho dos artefatos e matrizes de confusão.
- Investigar a robustez dos modelos sob ruído AWGN em múltiplos níveis de SNR e discutir limitações de generalização, vazamento de domínio e vieses de fonte.
- Disponibilizar um *pipeline* de treinamento, inferência e *benchmark* baseado em tecnologias de código aberto, com artefatos e figuras gerados de forma rastreável.

1.3 Questões de Pesquisa

Três questões orientam a avaliação experimental:

1. **QP1:** Como famílias heterogêneas de modelos — classificadores clássicos sobre descritores tabulares, redes espectro-temporais, redes de áudio bruto/grafos e *backbones* SSL congelados — se comparam sob um protocolo único de treinamento e avaliação em um *corpus* de fala em português?
2. **QP2:** Em que medida a acurácia em conjunto de teste limpo prediz a robustez sob ruído aditivo, e quais fatores (representação de entrada, capacidade do modelo) explicam eventuais dissociações?

3. **QP3:** Quais compromissos de custo computacional (latência, memória, número de parâmetros) acompanham cada família, e existe um modelo dominante em todas as dimensões?

As três questões são retomadas na análise dos resultados (Seção 5.3) e nas conclusões (Seção 6).

1.4 Organização do Trabalho

O restante do documento está organizado da seguinte forma: a Seção 2 revisa as tecnologias de síntese e manipulação de voz, a evolução dos métodos de detecção e o posicionamento deste trabalho em relação à literatura; a Seção 3 descreve as 11 arquiteturas comparadas; a Seção 4 detalha o *pipeline*, o *corpus*, as métricas e o protocolo de treinamento; a Seção 5 apresenta e analisa os resultados; e a Seção 6 sintetiza conclusões, contribuições, limitações e trabalhos futuros.

2 Tecnologias de Síntese de Voz

A síntese de voz reúne técnicas capazes de gerar ou transformar fala humana por meio de modelos acústicos, linguísticos e neurais. Nesta pesquisa, essas tecnologias são relevantes porque definem o tipo de artefato que o detector deve identificar: descontinuidades espectrais de *vocoders*, inconsistências prosódicas, vazamento de identidade vocal, ruído de canal ou padrões de codificação aprendidos por sistemas generativos. Assim, este capítulo separa as tecnologias de geração de fala dos métodos de detecção, preservando a ligação entre os ataques considerados e as arquiteturas avaliadas no restante do trabalho.

2.1 Da Síntese Clássica à Síntese Neural

Os primeiros sistemas eletrônicos de fala baseavam-se no modelo fonte-filtro, em que a excitação glotal e o trato vocal eram parametrizados separadamente [2, 3]. Posteriormente, a síntese concatenativa passou a combinar unidades gravadas de fala humana, obtendo maior naturalidade ao custo de grandes bases de áudio e menor flexibilidade prosódica [4]. A transição para modelos neurais substituiu regras explícitas por funções aprendidas: WaveNet modelou a forma de onda de modo autoregressivo [5], Tacotron e Tacotron 2 mapearam texto para espectrogramas Mel [6, 10], e arquiteturas posteriores como FastSpeech e VITS exploraram inferência mais rápida e geração *end-to-end* [19, 20].

Modelos recentes ampliaram esse cenário para síntese multilíngue, clonagem de voz e adaptação com poucas amostras. YourTTS e VALL-E exemplificam essa tendência ao combinar representação de locutor, conteúdo linguístico e codificadores neurais de áudio para generalizar a falantes não vistos [7, 21]. No conjunto utilizado neste trabalho, a fração Fake Voices é composta por amostras geradas com XTTS, sistema *cross-lingual* de TTS associado à classe sintética do *corpus* [22]. Essa origem justifica a avaliação de detectores capazes de lidar com fala neural em português brasileiro e com variações de locutor.

2.2 Text-to-Speech, Clonagem e Conversão de Voz

A síntese de fala por texto (*Text-to-Speech*, TTS) recebe uma sequência textual ou fonética e produz áudio inteligível. Em sistemas neurais clássicos, um modelo acústico transforma o texto em espectrograma Mel, que é convertido em forma de onda por um *vocoder* neural:

$$\mathbf{s}_{\text{mel}} = h_{\psi}(\mathbf{t}_{\text{text}}), \quad \mathbf{y}_{\text{audio}} = v_{\omega}(\mathbf{s}_{\text{mel}}) \quad (1)$$

em que $\mathbf{t}_{\text{text}}$ é o texto de entrada, $\mathbf{s}_{\text{mel}}$ é o espectrograma intermediário, $\mathbf{y}_{\text{audio}}$ é a forma de onda sintetizada, h_{ψ} representa o modelo acústico e v_{ω} representa o *vocoder*. A Equação 1 é uma abstração funcional do processo: implementações modernas podem empregar codificadores de fala, modelos auto-regressivos, difusão, *flow* ou variacionais, mas a separação entre conteúdo linguístico, representação acústica e síntese de onda permanece útil para

discutir artefatos detectáveis.

A clonagem de voz (*voice cloning*) consiste em replicar as características acústicas e prosódicas de um falante a partir de amostras limitadas [1]. Modelos como VALL-E [7] empregam abordagem *few-shot* baseada em codificadores neurais de áudio, com alta similaridade perceptual mesmo quando a referência de voz é curta:

$$\mathbf{y}_{\text{clone}} = f_{\theta}(\mathbf{x}_{\text{text}}, \mathbf{z}_{\text{speaker}}) \quad (2)$$

onde $\mathbf{y}_{\text{clone}}$ é o áudio sintético, $\mathbf{x}_{\text{text}}$ é o texto, $\mathbf{z}_{\text{speaker}}$ é o vetor de características do falante e f_{θ} é o modelo neural treinado. A Equação 2 evidencia que a ameaça não depende apenas da naturalidade da fala, mas também da preservação da identidade vocal, fator central em ataques contra biometria de voz.

A conversão de voz (*voice conversion*, VC) transforma a voz de um falante para soar como a de outro, preservando o conteúdo linguístico [11]. Diferentemente do TTS, a entrada já contém fala; o modelo busca separar conteúdo, prosódia e identidade para reemitir o sinal com características do falante-alvo:

$$\mathbf{y}_{\text{conv}} = g_{\phi}(\mathbf{x}_{\text{source}}, \mathbf{z}_{\text{target}}) \quad (3)$$

A Equação 3 formaliza esse mapeamento. Modelos como StarGAN-VC [23] usam redes adversariais para aprender mapeamentos muitos-para-muitos. A VC é particularmente desafiadora para detecção porque preserva boa parte do conteúdo linguístico e pode deslocar artefatos para regiões sutis do espectro, aumentando a dependência de modelos robustos a geradores não vistos [17].

A manipulação em tempo real representa a aplicação operacional da conversão de voz em chamadas, transmissões ou atendimento remoto. Em vez de assumir que o ataque seja imperceptível, a questão técnica relevante é que latências baixas reduzem a janela de verificação humana e automatizada [12]. Trata-se do mesmo mapeamento da Equação 3, executado sob restrição de latência de processamento (tipicamente dezenas de milissegundos por bloco de áudio): sistemas RVC e abordagens correlatas atendem a essa restrição com extratores de conteúdo leves e redes otimizadas para inferência contínua [11].

A Tabela 1 sintetiza as quatro modalidades descritas nesta seção, relacionando entrada, saída, artefatos acústicos esperados e o risco associado a cada uma.

Tabela 1: Principais modalidades de geração e manipulação de fala sintética.

Modalidade	Entrada típica	Saída	Artefatos esperados	Risco principal
TTS neural	Texto ou fonemas	Fala sintética completa	Prosódia artificial, marcas de <i>vocoder</i> , suavização espectral	Conteúdo falso atribuído a uma voz
Clonagem de voz	Texto e poucos segundos do locutor	Fala no timbre do alvo	Inconsistência de identidade, entonação e respiração	Fraudes de identidade
Conversão de voz	Fala de origem e identidade-alvo	Fala convertida	Artefatos de separação conteúdo/falante, fase e formantes	Impersonação em fala real
Manipulação em tempo real	Fluxo de áudio e perfil-alvo	Saída alterada em baixa latência	Compressão, instabilidade temporal e ruído de canal	Ataques em chamadas ao vivo

2.3 Implicações para a Detecção

As tecnologias descritas acima motivam detectores capazes de operar sobre representações complementares. Modelos baseados em descritores manuais capturam estatísticas acústicas agregadas; redes espectrais exploram padrões tempo-frequência; arquiteturas de áudio bruto modelam microestrutura temporal; e modelos auto-supervisionados herdam representações treinadas em grandes coleções de fala. A Tabela 2 sintetiza essa progressão histórica como evolução dos métodos de detecção, não como taxonomia de síntese.

Tabela 2: *Survey* das gerações de métodos de detecção de áudio sintético.

Geração	Representação	Classificador	Pontos Fortes	Limitações
1 ^a (até 2015)	MFCC, LFCC, CQCC	GMM-UBM, SVM [24]	Baixo custo computacional	Baixa generalização
2 ^a (2016–2019)	Espectrograma Mel	CNN, LSTM [25], LCNN	Robustez a artefatos de <i>vocoder</i>	Sensibilidade a geradores não vistos
3 ^a (2020–2022)	Forma de onda, grafos	RawNet2, AASIST	EER < 1% em base controlada	Alta demanda computacional (GPU)
4 ^a (2021+)	Representação SSL e atenção	WavLM, HuBERT, AST	Melhor aproveitamento de contexto global	Dependência de pré-treinamento e validação externa

Até o desafio ASVspoof 2015 [26], o estado da arte envolvia a extração manual de características paramétricas — como MFCC, LFCC e CQCC — seguida por classificadores

tradicionais. Sahidullah et al. [27] demonstraram que o LFCC supera o MFCC como *front-end* para detecção de *spoofing*, enquanto Todisco et al. [28] introduziram o CQCC como representação de referência. A partir do ASVspoof 2019 [15], CNNs, LCNNs [29], Res2Net [30] e modelos recorrentes passaram a explorar espectrogramas e mapas tempo-frequência. Em seguida, RawNet2 [31], AASIST [32] e RawGAT-ST aproximaram a detecção do sinal bruto e de grafos de atenção. Mais recentemente, WavLM [33], HuBERT [34], wav2vec 2.0 [35] e AST [36] incorporaram representações SSL e mecanismos de atenção inspirados por Transformers [37] e ViT [38]. Essa trajetória fundamenta a seleção das arquiteturas comparadas na Seção 3 e justifica a avaliação de modelos heterogêneos sob um mesmo protocolo.

2.4 Posicionamento em Relação à Literatura

A avaliação de contramedidas é hoje organizada em torno de desafios internacionais — as edições do ASVspoof [15–17, 26] e o ADD [18] — e de revisões sistemáticas recentes [9], majoritariamente centrados em inglês e mandarim. Para o português, os recursos públicos são recentes e ainda não acompanham um protocolo unificado de comparação: BRSpeech-DF [39] e Fake Voices [22] disponibilizam dados, mas não um *benchmark* padronizado entre famílias arquiteturais. Este trabalho posiciona-se nessa lacuna: não propõe uma nova arquitetura, mas submete 11 modelos heterogêneos a um mesmo protocolo de dados, treinamento e métricas — incluindo custo computacional e robustez a ruído — sobre fala em português. Diferentemente das tabelas de liderança do ASVspoof, o objetivo não é o estado da arte absoluto, e sim a comparabilidade controlada entre famílias sob condições idênticas.

3 Arquiteturas de Classificação

Esta seção descreve as 11 arquiteturas comparadas, agrupadas em três famílias segundo a representação de entrada consumida: modelos auto-supervisionados e de áudio bruto (Seção 3.1), redes neurais de domínio espectro-temporal (Seção 3.2) e classificadores estatísticos clássicos (Seção 3.3). Para cada modelo, apresentam-se a síntese arquitetural, o formato de entrada do artigo de referência e o escopo exato adotado neste trabalho.

3.1 Modelos Auto-Supervisionados (SSL) e de Áudio Bruto

Este grupo reúne modelos que evitam a dependência exclusiva de descritores tabulares extraídos manualmente. WavLM e HuBERT utilizam representações auto-supervisionadas aprendidas previamente em grandes coleções de fala, ao passo que RawNet2, AASIST e RawGAT-ST aprendem filtros e relações temporais diretamente a partir da forma de onda. Em todos os casos, o protocolo deste trabalho preserva a entrada em áudio bruto como eixo metodológico comum, adaptando apenas a cabeça supervisionada e os detalhes necessários para a execução reproduzível no *benchmark*.

WavLM [33]

Síntese Arquitetural: WavLM é um modelo SSL baseado em Transformer, treinado por predição mascarada de fala combinada a uma tarefa de denoising. Essa formulação busca preservar conteúdo linguístico, identidade do locutor e informações paralinguísticas, tornando o *backbone* útil para tarefas de fala além de reconhecimento automático.

Formato de entrada no artigo de referência: O modelo opera sobre forma de onda de fala, usualmente representada como sinal monofônico amostrado a 16 kHz. A extração de atributos é feita internamente pelo codificador convolucional e pelas camadas Transformer, sem necessidade de espectrogramas ou MFCCs pré-computados.

Escopo neste trabalho: O manifesto experimental avalia o WavLM Original por meio do *checkpoint* `microsoft/wavlm-base`, carregado via `PyTorch/transformers`. O *backbone* permanece congelado e o treinamento supervisionado ajusta apenas a cabeça de classificação binária sobre *embeddings* SSL extraídos de áudio bruto normalizado a 16 kHz. Essa escolha reduz custo computacional e evita confundir o efeito do pré-treinamento SSL com um ajuste fino integral do modelo. O qualificador “Original” distingue este uso do *checkpoint* pré-treinado real de eventuais implementações simplificadas; ele não implica ajuste fino completo, de modo que os resultados refletem a qualidade dos *embeddings* congelados, não o teto de desempenho do WavLM.

HuBERT (*Hidden-Unit BERT*) [34]

Síntese Arquitetural: HuBERT também pertence à família SSL, mas sua ideia central é prever unidades acústicas ocultas obtidas por agrupamento offline. O treinamento

aplica perda apenas sobre regiões mascaradas, forçando o modelo a aprender regularidades acústicas e linguísticas a partir de rótulos discretos pseudo-supervisionados.

Formato de entrada no artigo de referência: O modelo recebe fala em forma de onda contínua, sem extratores cepstrais externos. As unidades ocultas usadas como alvo de pré-treinamento são derivadas de agrupamentos sobre representações acústicas, mas a entrada operacional do modelo permanece o áudio bruto, tipicamente em 16 kHz.

Escopo neste trabalho: O HuBERT Original é avaliado com o *checkpoint facebook/hubert-base-ls960*, também via PyTorch/*transformers*. Assim como no WavLM, o *backbone* é mantido congelado e a etapa supervisionada treina uma cabeça de classificação sobre os *embeddings* extraídos. Com isso, HuBERT permanece no escopo principal dos 11 modelos consolidados, sem ser tratado como trabalho futuro.

RawNet2 [31]

Síntese Arquitetural: RawNet2 é uma rede *end-to-end* para anti-*spoofing* que substitui descritores manuais por filtros passa-banda aprendíveis. A arquitetura combina uma frente SincNet, blocos residuais com escalamento de mapas de características e uma camada GRU para agregação temporal antes da decisão binária.

Formato de entrada no artigo de referência: O artigo de referência processa diretamente a forma de onda bruta, sem converter previamente o sinal para espectrograma. Essa entrada preserva informações de fase, microvariações temporais e artefatos de síntese que podem ser suavizados por representações tempo-frequência.

Escopo neste trabalho: O benchmark utiliza a variante RawNet2 alinhada ao artigo (*paper-like*), operando sobre áudio bruto padronizado: frente SincNet, blocos residuais com escalamento de mapas de características (FMS, *Feature Map Scaling*), GRU e camada densa de 1024 unidades, com treinamento supervisionado em GPU sob o protocolo comum (Seção 4.7). O modelo representa a referência principal de aprendizagem direta sobre PCM no conjunto experimental.

AASIST [32]

Síntese Arquitetural: AASIST estende a família RawNet ao introduzir atenção em grafos espectrais e temporais. Após uma frente convolucional sobre a forma de onda, o modelo constrói subgrafos para capturar relações entre bandas frequenciais e segmentos temporais, integrando-os por uma camada heterogênea de atenção com nó de empilhamento.

Formato de entrada no artigo de referência: A entrada é áudio bruto. Os grafos espectro-temporais não são fornecidos como dados externos: eles são derivados internamente a partir das representações aprendidas pelo codificador inicial.

Escopo neste trabalho: A variante *aasist* é usada como implementação alinhada ao artigo, com SincConv, blocos residuais, ramos de GAT espectral e temporal, camadas

HS-GAL (*Heterogeneous Stacking Graph Attention Layer*), leitura por grafo e cabeça AM-Softmax (*Additive Margin Softmax*). O modelo é treinado sob o mesmo protocolo padronizado (Seção 4.7) e integra o grupo de modelos de áudio bruto/grafos do manifesto oficial.

RawGAT-ST [40]

Síntese Arquitetural: RawGAT-ST modela artefatos de falsificação por meio de redes de atenção em grafos aplicadas a dois domínios complementares: subbandas espectrais e intervalos temporais. A fusão em nível de modelo permite combinar evidências locais e globais sem depender de um *ensemble* externo de detectores.

Formato de entrada no artigo de referência: Assim como RawNet2 e AASIST, o RawGAT-ST recebe a forma de onda bruta. A estrutura espectro-temporal é aprendida a partir de um codificador inicial e depois organizada em subgrafos espectral e temporal.

Escopo neste trabalho: O benchmark utiliza uma variante RawGAT-ST com fusão multiplicativa e *stride* temporal igual a 4, composta por SincConv, encoder residual, GAT espectral, GAT temporal e fusão por multiplicação elemento a elemento. O *stride* temporal controla o custo quadrático da atenção em grafos, preservando o princípio arquitetural do artigo e viabilizando treinamento reproduzível em GPU.

3.2 Arquiteturas Neurais de Domínio Espectro-Temporal

Conformer [41]

Síntese Arquitetural: Topologia híbrida (*Convolution-augmented Transformer*) que integra a excelência de modelagem de contexto global da autoatenção com a eficiência na extração de padrões locais dos módulos convolucionais.

Desempenho e Observações: Demonstra alta precisão operando sobre representações bidimensionais densas (espectrogramas Mel), capturando artefatos de envoltória espectral e de textura tempo-frequência típicos da síntese por *vocoders* — a informação de fase, descartada na representação Mel de magnitude, não é modelada.

Escopo neste trabalho: O manifesto experimental usa a variante **conformer** sobre espectrogramas Mel, treinada do zero sob o protocolo comum (Seção 4.7), sem pré-treinamento em tarefas de reconhecimento de fala.

Compact Convolutional Transformer (CCT) [42]

Síntese Arquitetural: Variação otimizada do *Transformer* que substitui o fatiamento linear tradicional por uma etapa de tokenização convolucional precedente à inserção nos blocos de atenção.

Desempenho e Observações: Proporciona expressiva redução no custo paramétrico e na exigência de volume de dados, mitigando o custo quadrático da autoatenção sobre

mapas espectrais.

Escopo neste trabalho: A variante `cct` implementa a tokenização convolucional seguida de blocos de autoatenção sobre espectrogramas Mel; com $\approx 1,8$ milhão de parâmetros, é a menor rede espectral do recorte e serve de contraponto de custo aos Transformers maiores.

Audio Spectrogram Transformer (AST) [36]

Síntese Arquitetural: Adaptação direta do *Vision Transformer* (ViT) para o domínio acústico. Trata mapas tempo-frequência como sequências de *patches* processadas via autoatenção global pura.

Desempenho e Observações: Demanda calibragem minuciosa de hiperparâmetros e esquemas severos de regularização para evitar instabilidade quando treinado do zero. Neste trabalho, exibiu alta resiliência sob injeção de ruído aditivo (Seção 5.3).

Escopo neste trabalho: A variante `ast_vit_base_scratch` usa a topologia ViT-Base **treinada do zero** sobre espectrogramas Mel — diferentemente do artigo original, que parte de pesos pré-treinados em ImageNet/AudioSet. Os resultados refletem, portanto, a arquitetura em regime supervisionado puro, não o AST pré-treinado da literatura.

Res2Net (Topologia Multi-escala) [30]

Síntese Arquitetural: Evolução convolucional caracterizada pela introdução de conexões residuais hierárquicas de múltiplas escalas (resoluções) aninhadas dentro de um único bloco.

Desempenho e Observações: A extração multi-resolução permite ao modelo identificar simultaneamente artefatos de *spoofing* que se manifestam tanto em bandas largas quanto em nichos de alta frequência.

Escopo neste trabalho: O benchmark usa a variante `res2net50_scale4_no_se` (profundidade 50, fator de escala 4, sem blocos *squeeze-and-excitation*) sobre espectrogramas Mel, sob o mesmo protocolo padronizado das demais redes espectrais.

3.3 Classificadores Estatísticos Clássicos

Support Vector Machine (SVM) [24]

Síntese Arquitetural: Com *kernel* de base radial (RBF), o SVM é um classificador não linear que projeta vetores de características acústicas (por exemplo, LFCCs e CQCCs) em um espaço de maior dimensão, no qual otimiza o hiperplano de separação por maximização de margem entre classes.

Desempenho e Observações: Foi referência de desempenho nas primeiras edições de detecção de *spoofing* de voz (por exemplo, ASVspoof 2015), mas sua capacidade preditiva degrada de forma acentuada diante de ataques gerados por sistemas neurais recentes e,

sobretudo, em protocolos de generalização cruzada entre bases [43].

Escopo neste trabalho: O SVM é treinado sobre o vetor tabular de 63 descritores (estatísticas temporais, MFCC e RASTA-PLP; Seção 4.3) padronizado por *z-score* — e não sobre LFCC/CQCC, usados na literatura citada acima —, com busca em grade por validação cruzada de 5 dobras sobre o *kernel* (linear, RBF, polinomial), $C \in \{0,1; 1; 10; 100\}$ e $\gamma \in \{\text{scale}, \text{auto}, 10^{-3}-1\}$, seguida de reajuste final no conjunto de treino completo.

Random Forest [44]

Síntese Arquitetural: Método de aprendizagem em comitê (*ensemble learning*) baseado na agregação de múltiplas árvores de decisão estocásticas. O modelo reduz a propensão ao sobreajuste (*overfitting*) combinando amostragem *bootstrap* (*bagging*) com seleção aleatória de subespaços de características a cada nó de ramificação.

Desempenho e Observações: Destaca-se pela escalabilidade em arquiteturas multinúcleo e pela interpretabilidade direta da relevância dos atributos (*feature importance*). Por outro lado, tem dificuldade em capturar dependências espectro-temporais finas e assinaturas não estacionárias relevantes para a detecção de *deepfakes* de voz mais recentes (padrão ASVspoof 2019 e sucessores) — limitação também observada empiricamente neste trabalho na robustez sob ruído (Seção 5.3).

Escopo neste trabalho: O *Random Forest* consome o mesmo vetor tabular de 63 descritores do SVM, com busca em grade por validação cruzada de 5 dobras sobre o número de árvores ($\{200, 300\}$), a profundidade máxima ($\{10, 15, 20\}$), os mínimos de divisão e de folha ($\{5, 10, 20\}$ e $\{2, 4, 8\}$) e o número de características por nó (*sqrt*, *log2*); a grade restringe deliberadamente a profundidade máxima e exige folhas maiores, como regularização estrutural contra o sobreajuste.

4 Metodologia

4.1 Visão Geral do Pipeline

O sistema proposto opera em duas fases: **treinamento** e **predição**. Durante o treinamento, o modelo aprende a distinguir características de voz real e sintética. Na predição, processa novos áudios e fornece classificação binária (real/sintético) com pontuação de confiança.

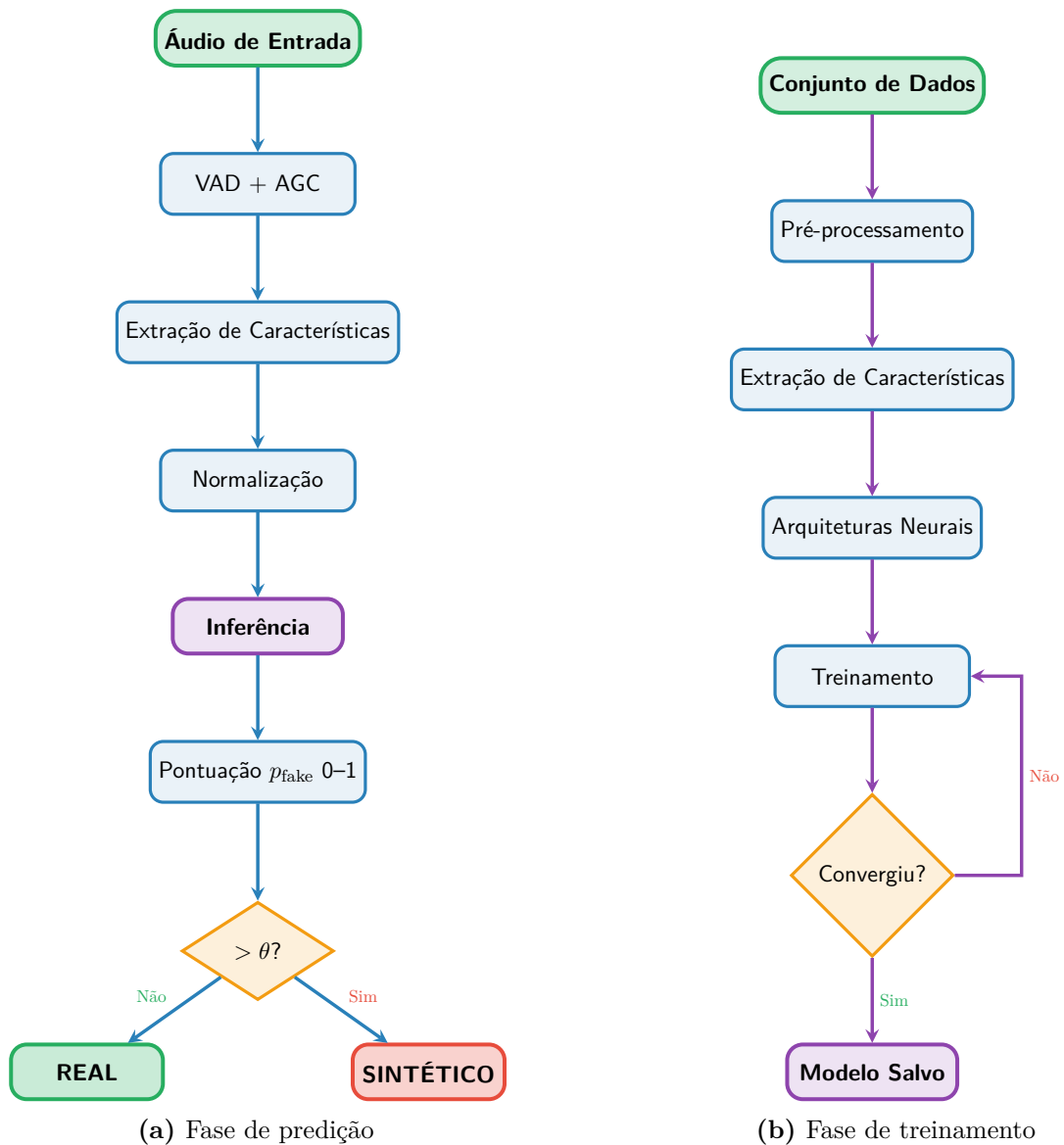


Figura 1: Diagrama do *pipeline* proposto. O modelo treinado (b) é implantado (*deploy*) na fase de inferência (a). A pontuação p_{fake} é a probabilidade predita de a amostra ser sintética: valores acima do limiar de decisão θ resultam em SINTÉTICO. O limiar é 0,5 por padrão; para WavLM Original e HuBERT Original, é calibrado em validação com ruído ($\theta \approx 0,48$; Seção 4.5).

4.2 Pré-processamento: Normalização e Detecção de Atividade de Voz

O bloco “VAD + AGC” da Figura 1 padroniza o áudio de entrada antes da extração de características, cobrindo múltiplos formatos (WAV, MP3, FLAC, OGG) com conversão automática para 16 kHz mono, PCM linear de 16 bit.

Normalização por Controle Automático de Ganho (AGC)

O nível de amplitude é normalizado para um alvo de *loudness* fixo:

$$x_{\text{norm}}[n] = x[n] \cdot \frac{A_{\text{alvo}}}{\sqrt{\frac{1}{N} \sum_{i=0}^{N-1} x[i]^2}}, \quad A_{\text{alvo}} = 10^{L_{\text{alvo}}/20} \quad (4)$$

em que N é o número total de amostras do sinal, $L_{\text{alvo}} = -23$ é o nível alvo de *loudness* (em LUFS) e A_{alvo} é a amplitude linear correspondente, referida ao fundo de escala unitário. A Equação 4 implementa uma aproximação por normalização RMS: a conversão LUFS exata exigiria ponderação *K-weighting* (ITU-R BS.1770), não implementada nesta etapa.

Detecção de Atividade de Voz (VAD)

O VAD por energia remove segmentos silenciosos ou não vocalizados, calculando a energia em decibéis por janela deslizante:

$$E[m] = 10 \log_{10} \left(\frac{1}{N_w} \sum_{n=0}^{N_w-1} |x[mH + n]|^2 \right) \quad (5)$$

$$\text{Voice}[m] = \begin{cases} 1, & \text{se } E[m] > \theta_{VAD} \\ 0, & \text{caso contrário} \end{cases} \quad (6)$$

em que N_w é o comprimento da janela de análise (25 ms), H é o *hop length* (passo, em amostras, entre janelas consecutivas; a 16 kHz, corresponde a 10 ms) e $\theta_{VAD} = -25$ dB é o limiar de energia, referido ao fundo de escala do sinal normalizado (dBFS). Em uma etapa de pós-filtragem, segmentos contíguos marcados como fala com duração inferior a $D_{\text{min}} = 0,3$ s são descartados, evitando a retenção de transientes isolados (Equações 5–6).

VAD em Ambiente de Produção

O Silero VAD (disponibilizado via Torch Hub) oferece performance pré-treinada superior ao VAD por energia. Nos experimentos, a indisponibilidade do `torchaudio` motivou o uso do VAD por energia como *fallback*, preservando a funcionalidade do *pipeline* sem dependências externas não atendidas no ambiente de CPU.

4.3 Representações de Entrada e Características Acústicas

O *pipeline* adapta a representação de entrada a cada família de arquitetura, preservando o mesmo áudio pré-processado ($\mathbf{x} \in \mathbb{R}^{80000 \times 1}$, 16 kHz mono, 5s) como origem comum. A Tabela 3 resume essa correspondência.

Tabela 3: Representação de entrada consumida por família de modelo.

Representação	Modelos	Observação
Áudio bruto (<i>raw-audio</i>)	WavLM, HuBERT, RawNet2, AASIST, RawGAT-ST	Forma de onda com <i>z-score</i> por amostra, sem extrator externo; RawNet2/AASIST/RawGAT-ST consomem recorte central de 1 s (16.000 amostras), WavLM/HuBERT a janela integral
Espectrograma Mel	Conformer, CCT, AST, Res2Net	Mapa 100×80 em dB (ref. máx.) com <i>z-score</i> por amostra (Equação 9)
Vetor tabular (63 descritores)	SVM, <i>Random Forest</i>	Estatísticas de amplitude, MFCC e RASTA-PLP

Para os classificadores clássicos, a ausência de camadas convolucionais ou de atenção exige um vetor de características de dimensão fixa. O módulo `benchmarks/data.py` constrói, para cada amostra, um vetor de 63 descritores compostos por três famílias:

- **Estatísticas temporais (11):** média, desvio-padrão, média absoluta, valor RMS, mínimo, máximo, percentis 25/50/75, energia da diferença entre amostras consecutivas (proxy de rugosidade do sinal) e taxa de cruzamento por zero (ZCR).
- **MFCC (26):** média e desvio-padrão de 13 coeficientes cepstrais na escala Mel por amostra.
- **RASTA-PLP (26):** média e desvio-padrão de 13 coeficientes RASTA-PLP (*Relative SpecTrA – Perceptual Linear Prediction* [45]) por amostra. O filtro RASTA atenua variações espectrais lentas associadas a efeitos de canal e foi incorporado especificamente para mitigar a degradação de SVM e *Random Forest* sob ruído, discutida na Seção 5.

Os coeficientes cepstrais na escala Mel (MFCC) que compõem a segunda família são obtidos por:

$$MFCC[m, c] = \sum_{k=0}^{M-1} \log(S[m, k]) \cdot \cos\left(\frac{\pi c(k + 0,5)}{M}\right) \quad (7)$$

em que $S[m, k]$ são as energias dos $M = 40$ filtros triangulares na escala Mel para o quadro m , e $c \in \{0, \dots, 12\}$ indexa os 13 coeficientes efetivamente extraídos na Equação 7 (média e desvio-padrão dos quais formam os 26 descritores da família MFCC). Antes do ajuste dos classificadores clássicos, cada dimensão do vetor de 63 descritores é padronizada por

normalização estatística (*z-score*):

$$X_{norm} = \frac{X - \mu}{\sigma} \quad (8)$$

em que μ e σ são a média e o desvio-padrão de cada característica, estimados sobre o conjunto de treino (`StandardScaler`, `scikit-learn`) e reaplicados sem reajuste sobre validação e teste.

Esse vetor de 63 dimensões implementa, de forma concreta, os descritores estatísticos temporais e cepstrais mencionados nos objetivos deste trabalho; a contribuição relativa de cada descritor para a decisão do *Random Forest* treinado é quantificada empiricamente na Seção 5.3 (Figura 10). Assim, os 11 modelos comparados neste trabalho cobrem três regimes de entrada — áudio bruto, espectrograma Mel e vetor tabular temporal/cepstral; os descritores adicionais implementados no repositório, mas não consumidos nesta rodada de *benchmark*, são resumidos adiante nesta seção.

Espectrograma Mel

A representação tempo-frequência consumida por Conformer, CCT, AST e Res2Net (Tabela 3) é computada por:

$$MelSpec[m, k] = 10 \log_{10} \left(\frac{\sum_f H_k[f] \cdot |STFT[m, f]|^2 + \varepsilon}{\max_{m', k'} P[m', k']} \right) \quad (9)$$

em que f indexa as *bins* de frequência, $H_k[f]$ é o banco de 80 filtros triangulares na escala Mel, $STFT$ é a transformada de Fourier de curto termo com janela de 512 amostras (n_fft), ε evita $\log(0)$ e o denominador referencia a escala em decibéis ao máximo do próprio mapa de potência P da amostra (Equação 9). O *hop length* é dimensionado dinamicamente para produzir exatamente 100 quadros por janela de 5 s (80.000 amostras $\Rightarrow$ *hop* de 800), resultando em mapas de dimensão fixa 100×80 , padronizados em seguida por normalização *z-score* por amostra — mesma normalização aplicada, na entrada em áudio bruto, aos modelos AASIST, RawGAT-ST e RawNet2, que consomem um **recorte central de 1 s** (16.000 amostras) da janela de 5 s para conter o custo quadrático da atenção em grafos e o comprimento da sequência da GRU.

Descritores Adicionais Disponíveis no Pipeline

Além dos regimes de entrada efetivamente comparados neste *benchmark*, o repositório implementa extratores adicionais como adaptadores `IFeatureExtractor` (`app/domain/features/extractors/`), utilizáveis em extensões futuras do protocolo (Seção 6.3) mas não consumidos pelos 11 modelos deste trabalho.

Descritores de forma espectral. Centroide, *rolloff*, largura de banda, taxa de cruzamento por zero (ZCR), contraste e planura espectrais caracterizam a distribuição de

energia no domínio da frequência:

$$SC[m] = \frac{\sum_{k=0}^{K-1} k \cdot |X[m, k]|^2}{\sum_{k=0}^{K-1} |X[m, k]|^2} \quad (\text{centroide}) \quad (10)$$

$$SR[m] = k_r \text{ tal que } \sum_{k=0}^{k_r} |X[m, k]|^2 = 0,85 \sum_{k=0}^{K-1} |X[m, k]|^2 \quad (\text{rolloff a 85\%}) \quad (11)$$

$$SB[m] = \sqrt{\frac{\sum_{k=0}^{K-1} (k - SC[m])^2 \cdot |X[m, k]|^2}{\sum_{k=0}^{K-1} |X[m, k]|^2}} \quad (\text{largura de banda}) \quad (12)$$

$$ZCR[m] = \frac{1}{2N} \sum_{n=0}^{N-1} |\text{sgn}(x[mH + n]) - \text{sgn}(x[mH + n - 1])| \quad (13)$$

$$SFlatness[m] = \frac{\left(\prod_{k=0}^{K-1} |X[m, k]|^2 \right)^{1/K}}{\frac{1}{K} \sum_{k=0}^{K-1} |X[m, k]|^2} \quad (14)$$

em que $X[m, k]$ é o espectro de magnitude do quadro m na *bin* de frequência k , entre K *bins* totais.

Constant-Q Transform (CQT). Alternativa de resolução log-frequencial ao espectrograma linear, usada por Todisco et al. como *front-end* de referência para *anti-spoofing* [28]:

$$CQT[m, k] = \sum_n x[n] \cdot g_k[n - mH] \cdot e^{-j2\pi Q_k n / N_k} \quad (15)$$

com 12 *bins* por oitava e 7 oitavas (84 características) na implementação disponível.

Descritores prosódicos e de qualidade vocal. Complementam os regimes acima com informação de excitação glotal — frequência fundamental (F0), *jitter* e *shimmer* (perturbação ciclo-a-ciclo de período e amplitude) e HNR (*Harmonic-to-Noise Ratio*):

$$Jitter = \frac{1}{N-1} \sum_{i=1}^{N-1} \frac{|T_i - T_{i-1}|}{\bar{T}} \quad (16)$$

$$Shimmer = \frac{1}{N-1} \sum_{i=1}^{N-1} \frac{|A_i - A_{i-1}|}{\bar{A}} \quad (17)$$

$$HNR = 10 \log_{10} \left(\frac{P_{\text{harmonic}}}{P_{\text{noise}}} \right) \quad (18)$$

em que T_i e A_i são o período e a amplitude do i -ésimo ciclo glotal, e $P_{\text{harmonic}}/P_{\text{noise}}$ são as potências das componentes harmônica e não-harmônica do sinal.

Características *delta* e *delta-delta*. Capturam a dinâmica temporal de primeira e

segunda ordem de qualquer descritor de quadro $X[m, c]$:

$$\Delta X[m, c] = \frac{\sum_{n=1}^N n(X[m+n, c] - X[m-n, c])}{2 \sum_{n=1}^N n^2} \quad (19)$$

A Tabela 4 resume esses descritores por tipo, dimensionalidade e validação na literatura de *anti-spoofing*, cuja hierarquia discriminativa motivou a escolha do vetor tabular efetivamente avaliado: características espectrais e cepstrais apresentam desempenho consistentemente superior a descritores prosódicos isolados, conforme observado nas edições do ASVspoof [15, 27, 28] e no desafio ADD [18].

Tabela 4: Descritores acústicos disponíveis no *pipeline*: síntese da literatura. **Custo Computacional:** estimativa qualitativa (Baixo/Médio/Alto) do consumo de tempo e memória para extração.

Tipo	Dimen.	Custo Comp.	Validação na Literatura
Espectrais (forma)	6	Médio	Sensíveis a artefatos de <i>vocoder</i> na envoltória espectral [28]
MFCC	13	Baixo	Referência clássica; LFCC supera MFCC em <i>spoofing</i> desde 2015 [15, 27]
Espectrograma Mel	80 bandas	Baixo	Base para CNNs/ <i>Transformers</i> ; captura artefatos de síntese tempo-frequência [31, 32]
CQT	84	Alto	Resolução logarítmica; CQCC como <i>front-end</i> robusto [28]
Prosódicas / qualidade vocal	6	Médio	Desempenho isolado inferior; fusão com espectrais melhora resultados [18]

4.4 Conjunto de Dados e Protocolo Experimental

4.4.1 Composição e Proveniência

O protocolo experimental desta pesquisa fundamenta-se em um *corpus* composto por 15.000 amostras de áudio em português, balanceado entre as classes (50% *bonafide* e

Tabela 5: Composição analítica da base de dados (15.000 amostras) estratificada por classe e proveniência estrutural. A disjunção de locutores entre as partições de treino/validação/teste não é garantida (Seção 4.4.3).

Classe	Fonte Original	Volume (Amostras)	Identificação do Locutor
<i>Bonafide</i>	BRSpeech-DF	3.750	Não individualizada
<i>Bonafide</i>	MLS Portuguese	1.875	21 locutores identificados
<i>Bonafide</i>	TTS-Portuguese Corpus	1.875	Locutor único
<i>Spoof</i>	BRSpeech-DF	3.750	Não individualizada
<i>Spoof</i>	Fake Voices (XTTS)	3.750	50 locutores identificados

Tabela 6: Especificações técnicas do protocolo de pré-processamento acústico e partições de amostragem.

Parâmetro Metodológico	Especificação Nominal
Pré-processamento Acústico	
Taxa de amostragem	16 kHz
Formato e Resolução quantizada	Monofônico, PCM linear de 16 bit
Janelamento temporal canônico	5 s (ajustado via truncamento ou <i>zero-padding</i>)
Dimensionalidade do vetor	$\mathbf{x} \in \mathbb{R}^{80.000 \times 1}$ (domínio do tempo discreto)
Representações de entrada	<i>Raw-audio</i> , Espectrogramas Mel ou vetor tabular (MFCC + RASTA-PLP)
Particionamento Estratificado	
Distribuição dos subconjuntos	70 % Treinamento, 15 % Validação, 15 % Teste
Cardinalidade por partição	10.500 (Tr), 2.250 (Vl), 2.250 (Te)
Avaliação de robustez (Ruído)	Injeção de AWGN (<i>SNR</i> de 30, 20 e 10 dB)

50 % *spoof*). A classe genuína (*bonafide*) reúne subconjuntos provenientes do BRSpeech-DF [39], amostras de leitura do *Multilingual LibriSpeech* (MLS) Portuguese [46] e o *TTS-Portuguese Corpus* [47].

A classe artificial (*spoof*) agrega a fração sintética do BRSpeech-DF e amostras geradas por XTTS no *corpus* Fake Voices [22]. A Tabela 5 consolida a proveniência dos dados, discriminando a variabilidade de locutores e as estratégias de amostragem empregadas.

4.4.2 Pré-processamento e Particionamento

Para assegurar homogeneidade no espaço de características acústicas, estabeleceu-se um fluxo de padronização de sinais e amostragem estratificada. Esse procedimento preserva a proporção de 1 : 1 entre as classes em todas as partições experimentais, mitigando vieses probabilísticos *a priori*. A estratificação atua apenas sobre o rótulo de classe: a proporção de fontes por partição não é explicitamente controlada, limitação relevante à luz da Seção 4.4.3. Os parâmetros operacionais desse fluxo estão formalizados na Tabela 6.

A matriz unidimensional resultante ($\mathbf{x} \in \mathbb{R}^{80.000 \times 1}$) atua como estrutura canônica de

entrada tanto para o processamento direto ponta a ponta (*end-to-end*) quanto para os módulos extratores de representações tempo-frequência. Em paralelo, a inserção controlada de Ruído Branco Gaussiano Aditivo (AWGN) no conjunto de teste simula uma degradação acústica simples e reproduzível, permitindo mensurar a resiliência dos classificadores sob diferentes níveis de SNR. O ruído é injetado, com semente fixa, no espaço de representação consumido por cada família: na forma de onda para os modelos de áudio bruto e SSL, sobre o espectrograma log-Mel para as redes espectrais e sobre o vetor tabular para os classificadores clássicos — escolha deliberada que uniformiza o teste entre arquiteturas, ao custo de, nos dois últimos casos, aproximar (e não reproduzir exatamente) o efeito do AWGN acústico sobre o sinal.

4.4.3 Limitações do Protocolo: Viés de Canal e Sobreposição de Locutores

O protocolo apresenta duas limitações estruturais: a ausência de disjunção estrita de locutores (*speaker-disjoint protocol*) e o pareamento assimétrico das fontes de dados. Em partições sem metadados individualizados de identidade — caso do BRSpeech-DF —, há risco real de sobreposição (*overlap*) de vozes entre os subconjuntos de treinamento e teste.

Além disso, a escassez de amostras acusticamente pareadas (*twins*) — uma vez que o *corpus* Fake Voices não tem contrapartidas autênticas, e os *datasets* MLS e TTS-Portuguese não têm derivações sintéticas correspondentes — introduz variáveis de confusão (*confounding variables*) relevantes. Essa assimetria entre fontes favorece o vazamento de domínio (*domain leakage*): os modelos podem otimizar a decisão a partir de perfis espúrios de codificação ou ruído de canal, em vez de aprender apenas assinaturas do processo de síntese de voz.

Consequentemente, as métricas reportadas na Seção 5 caracterizam eficácia preditiva em distribuição fechada (*in-domain*) e exigem cautela quanto à validade externa. A atenuação desses vieses por meio de partições estritamente disjuntas e validação cruzada entre bases (*cross-dataset*) constitui uma prioridade para investigações subsequentes (Seção 6.3).

4.5 Métricas de Avaliação

O protocolo experimental reporta um conjunto padronizado de métricas de classificação binária (*bonafide* vs. *spoof*), calculadas a partir da pontuação de *spoof* $p_{fake} \in [0, 1]$ predita por cada modelo.

Acurácia, Precisão, Revocação e F1-score

Sejam TP , TN , FP e FN os verdadeiros positivos, verdadeiros negativos, falsos positivos e falsos negativos, respectivamente, obtidos ao limiar de decisão θ ($p_{fake} > \theta$):

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN}, \quad F1 = 2 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}} \quad (20)$$

em que $\text{Precisão} = TP/(TP + FP)$ e $\text{Revocação} = TP/(TP + FN)$ na Equação 20.

Para 9 dos 11 modelos, o limiar é o padrão $\theta = 0,5$. Para WavLM Original e HuBERT Original, cujo treinamento da cabeça de classificação inclui calibração de ponto de operação (Seção 5.3), θ é fixado no limiar de EER estimado sobre a validação com ruído ($\theta \approx 0,48$ em ambos os casos — 0,483 e 0,482, respectivamente —, persistido junto aos artefatos). Essa assimetria é declarada por transparência: ela afeta apenas as métricas dependentes de limiar (acurácia, precisão, revocação e F1) e em magnitude pequena, dado que os limiares calibrados distam menos de 0,02 do padrão; AUC-ROC, EER e *min t*-DCF varrem todos os limiares e não são afetados.

AUC-ROC

A curva ROC (*Receiver Operating Characteristic*) relaciona a taxa de verdadeiros positivos (TPR) à taxa de falsos positivos (FPR) variando-se o limiar de decisão θ sobre p_{fake} . A AUC-ROC é a área sob essa curva: valor 1,0 indica separação perfeita entre as classes e 0,5 equivale a uma decisão aleatória.

Taxa de Erro Igual (EER)

O EER é o ponto de operação θ^* em que a taxa de falsos positivos iguala a taxa de falsos negativos ($FNR = 1 - TPR$):

$$\text{EER} = FPR(\theta^*), \quad \text{tal que} \quad FPR(\theta^*) = FNR(\theta^*). \quad (21)$$

Na Equação 21, valores menores indicam maior separabilidade entre as distribuições de pontuação das classes *bonafide* e *spoof*, independentemente da escolha de limiar.

t-DCF (proxy interno)

O *t*-DCF (*Tandem Detection Cost Function*) normalizado, proposto por Kinnunen et al. [48] para o protocolo ASVspoof 2019, pondera em conjunto os custos de falha do detector (*countermeasure*, CM) e de um sistema de verificação de locutor (ASV) operando em tandem:

$$t\text{-DCF}(\theta) = \frac{C_1 \cdot P_{\text{miss}}^{\text{cm}}(\theta) + C_2 \cdot P_{\text{fa}}^{\text{cm}}(\theta)}{\min(C_1, C_2)} \quad (22)$$

com os coeficientes de custo dados por

$$C_1 = P_{\text{tar}}(C_{\text{miss}} - C_{\text{miss}} P_{\text{miss}}^{\text{asv}}) - P_{\text{non}} C_{\text{fa}} P_{\text{fa}}^{\text{asv}}, \quad C_2 = C_{\text{fa}} P_{\text{spoof}}(1 - P_{\text{miss,spoof}}^{\text{asv}}). \quad (23)$$

Na Equação 23, $P_{\text{miss}}^{\text{cm}}(\theta)$ é a probabilidade de o detector rejeitar (classificar como *spoof*) uma amostra *bonafide*, e $P_{\text{fa}}^{\text{cm}}(\theta)$ é a probabilidade de aceitar (classificar como *bonafide*) uma amostra *spoof*; o *min t-DCF* reportado nas tabelas minimiza a Equação 22 sobre θ . Este trabalho não dispõe de um sistema ASV pareado ao *corpus* experimental: os parâmetros do ASV ($P_{\text{fa}}^{\text{asv}} = 0,0524$, $P_{\text{miss}}^{\text{asv}} = 0,0104$, $P_{\text{miss,spoof}}^{\text{asv}} = 0,0058$) e as prioris ($P_{\text{tar}} = 0,9405$, $P_{\text{non}} = 0,0095$, $P_{\text{spoof}} = 0,05$, $C_{\text{miss}} = 1$, $C_{\text{fa}} = 10$) são fixados nos valores padrão do particionamento LA do ASVspoof 2019 [48], e apenas o termo do detector ($P_{\text{miss}}^{\text{cm}}$, $P_{\text{fa}}^{\text{cm}}$) é medido sobre os modelos deste *benchmark*. Por essa razão, o valor reportado é tratado como **proxy interno** — comparável entre os 11 modelos deste trabalho, mas não equivalente ao *min t-DCF* oficial de um protocolo com sistema ASV real — e identificado com asterisco (*t-DCF**) nas tabelas da Seção 5.

4.6 Ferramentas de Software Livre

Para garantir reprodutibilidade e escalabilidade, a arquitetura foi desenvolvida com ferramentas de código aberto e bibliotecas amplamente adotadas pela comunidade científica, conforme descrito a seguir:

- **Pré-processamento e Detecção de Atividade de Voz (VAD):** O *pipeline* integra o modelo *Silero VAD* (via *Torch Hub*) com mecanismo de contingência (*fallback*) baseado em limiar de energia; nos experimentos deste trabalho, dada a indisponibilidade do ecossistema *torchaudio* no ambiente de execução, utilizou-se o VAD por energia descrito na Seção 4.2. As operações de controle automático de ganho (AGC) e reamostragem foram gerenciadas pela biblioteca *librosa*, assegurando eficiência computacional em CPU.
- **Extração de Características Acústicas:** A biblioteca *librosa* foi incorporada para a extração de estatísticas temporais e coeficientes cepstrais na escala Mel (MFCC); os coeficientes RASTA-PLP são calculados pelo extrator próprio do repositório (Seção 4.3).
- **Modelos de Fundação (*Foundation Models*):** WavLM e HuBERT foram integrados via ecossistema *transformers* (Hugging Face). Neste protocolo, os *backbones* permanecem congelados e apenas a cabeça de classificação é treinada sobre os *embeddings* extraídos.
- **Arquiteturas Especializadas:** As redes RawNet2, AASIST e RawGAT-ST foram reimplementadas em TensorFlow/Keras seguindo as implementações de referência pu-

blicadas pelos autores em PyTorch (licença MIT), preservando os blocos arquiteturais descritos na Seção 3.1.

- **Treinamento e Classificação:** O arcabouço TensorFlow/Keras foi utilizado no desenvolvimento e treinamento das arquiteturas neurais espectrais e de áudio bruto; WavLM Original e HuBERT Original usam um *runner* dedicado em PyTorch/`transformers`; os classificadores clássicos foram implementados com Scikit-Learn.
- **Mecanismos de Interpretabilidade (Propostos):** Prevê-se a integração da biblioteca `shap` para o cálculo de valores SHAP [49] e do framework `captum` (PyTorch) para a geração de mapas de ativação via GradCAM, planejados como extensões metodológicas subsequentes (Seção 6.3).
- **Disponibilidade:** o código-fonte completo do *pipeline*, os *scripts* de treinamento e *benchmark* e as configurações utilizadas estão publicamente disponíveis em <https://github.com/thierrybraga/XFakeSong>, sob licença MIT.

4.7 Protocolo de Treinamento

O regime de treinamento aplicado às arquiteturas conexionistas consistiu em um protocolo padronizado com teto de 100 épocas para as redes espectrais (Conformer, CCT, AST, Res2Net) e RawNet2, e de 120 épocas para AASIST, RawGAT-ST e para as cabeças de classificação de WavLM/HuBERT Original. A parada antecipada monitora a perda de validação, com paciência de 15 a 25 épocas conforme o modelo (Tabela 13) e salvamento automatizado do melhor *checkpoint* com restauração dos melhores pesos; o RawNet2 é a exceção, treinando o orçamento integral de 100 épocas com seleção do melhor *checkpoint*. Todos os modelos neurais foram treinados com *augmentation* de ruído aditivo sobre a representação de entrada (no caso das cabeças SSL, com SNR sorteado em {30, 20, 10, 5} dB); para SVM e *Random Forest*, o conjunto de treino é ampliado com cópias contaminadas por AWGN nos mesmos níveis de SNR avaliados no teste (30/20/10 dB), aplicadas no espaço de características tabular. O número de épocas efetivamente treinadas por modelo — tipicamente inferior ao teto por ação da parada antecipada — é reportado na coluna *Treino* da Tabela 8. Durante o processo, coletaram-se históricos de perda (*loss*) e acurácia, matrizes de confusão, curvas ROC e de Precisão-Revocação (*Precision-Recall*), métricas por classe, tamanho em disco e latência de inferência — esta última medida como a mediana de 30 inferências consecutivas de amostra única, após duas execuções de aquecimento, no ambiente de execução de cada modelo (Seção 5.1). Todos os experimentos usam semente aleatória fixa (42) para particionamento, inicialização e geração de ruído, registrada nos artefatos de cada execução.

Para os classificadores estatísticos clássicos, realizou-se busca em grade de hiperparâmetros (*hyperparameter grid search*) antes do ajuste final. No cálculo geral, avaliaram-se

11 modelos consolidados: Random Forest, SVM, CCT, AST, Res2Net, Conformer, RawNet2, AASIST, RawGAT-ST, WavLM Original e HuBERT Original. Extensões de fusão tardia via *ensemble* adaptativo permanecem delineadas na Seção 6.3.

Tabela 7: Especificações de infraestrutura, hardware e parametrização do ambiente de *benchmarking*.

Componente de Infraestrutura	Configuração e Parâmetros Operacionais
Sistema operacional base	Windows 11 + WSL2 (Ambiente nativo Linux Ubuntu)
Fase de treinamento	Execução em GPU com precisão mista e controle de VRAM
Ambiente de avaliação de inferência	Heterogêneo por modelo: CPU (CCT, Conformer, SVM, RF) ou GPU RTX 3060 (demais)
Modelos conexionistas profundos	Abstração TensorFlow/Keras com persistência de <i>checkpoints</i>
Modelos baseados em SSL	WavLM Original e HuBERT Original via transformers
Modelos estatísticos clássicos	Processamento multi-core via Scikit-Learn com validação cruzada
Volume do <i>corpus</i> consolidado	15.000 amostras balanceadas (7.500 reais / 7.500 sintéticas)
Particionamento de dados (<i>Split</i>)	Razão 70/15/15 (10.500 treino, 2.250 validação, 2.250 teste)
Especificação padrão de sinal	Monofônico, 16 kHz, 5 s ($\mathbf{x} \in \mathbb{R}^{80.000 \times 1}$ em <i>raw-audio</i>)

5 Experimentos e Resultados

5.1 Ambiente Experimental e Hardware

O ambiente computacional dedicado à execução do *benchmark* foi estruturado sobre *Windows Subsystem for Linux 2* (WSL2), em Windows 11, com apoio de contêineres Docker para isolar dependências e facilitar a reprodutibilidade. Na fase de treinamento, o *pipeline* foi configurado com suporte a processamento gráfico (GPU), precisão mista (*mixed precision*) e controle de alocação de memória de vídeo (VRAM) para otimizar a vazão dos lotes (*batch size*).

As métricas de inferência foram extraídas no ambiente de execução de cada modelo, utilizando TensorFlow 2.21.0 sobre Python 3.11.15: CCT, Conformer, SVM e *Random Forest* em CPU; AST, Res2Net, RawNet2, AASIST e RawGAT-ST em ambiente com GPU disponível; e WavLM/HuBERT Original em GPU (CUDA), via *runner* PyTorch. Essa heterogeneidade de dispositivos, registrada nos artefatos de cada execução, impede a comparação direta de latência entre famílias: as latências da Tabela 9 devem ser lidas como ordens de grandeza por dispositivo, e não como um *ranking* único (Seção 5.3). Os classificadores estatísticos clássicos foram processados com paralelismo multi-core e validação cruzada. O sumário das configurações de hardware e infraestrutura consta na Tabela 7.

5.2 Resultados Consolidados

Os resultados oficiais, tabelas de eficiência, robustez, estabilidade, gráficos comparativos e matrizes de confusão são incorporados a partir do fragmento gerado automaticamente em `tabelas_benchmark.tex`. Esse procedimento evita divergência entre os artefatos experimentais e o texto principal. O fragmento constitui a fonte única para as Tabelas 8, 9, 10 e 11, além das Figuras 2, 3, 4, 5, 6, 7, 8 e 9. O protocolo de treinamento e as ferramentas de software que geraram esses artefatos estão descritos nas Seções 4.6 e 4.7.

Tabela 8: Resultados consolidados no conjunto de teste limpo.

Modelo	Acur.	EER	t -DCF*	AUC	F1	Lat. (ms)	Treino	Status
Random Forest	97,82%	2,09%	0,0409	0,999	97,83%	36,53	CV+fit	OK
SVM	99,24%	0,58%	0,0176	1,000	99,24%	0,28	CV+fit	OK
CCT	99,20%	0,71%	0,0196	0,999	99,20%	41,61	100	OK
AST	99,02%	0,98%	0,0265	0,998	99,02%	51,79	100	OK
Res2Net	99,69%	0,36%	0,0069	1,000	99,69%	44,58	100	OK
Conformer	99,82%	0,18%	0,0034	1,000	99,82%	45,34	100	OK
RawNet2	97,16%	2,71%	0,0624	0,998	97,11%	42,33	100	OK
AASIST	95,02%	4,89%	0,1267	0,990	95,10%	53,86	100	OK
RawGAT-ST	93,60%	6,22%	0,1745	0,986	93,87%	33,37	100	OK
WavLM Original	99,69%	0,36%	0,0062	1,000	99,69%	18,73	100	OK
HuBERT Original	99,87%	0,18%	0,0027	1,000	99,87%	17,00	100	OK

t -DCF*: proxy interno de custo normalizado, sem acoplamento a um sistema ASV oficial.

Tabela 9: Eficiência computacional dos modelos consolidados.

Modelo	Parâmetros	Tam. (MB)	Lat. (ms)	Acur.	EER
Random Forest	—	46,45	36,53	97,82%	2,09%
SVM	—	2,01	0,28	99,24%	0,58%
CCT	1.822.479	7,15	41,61	99,20%	0,71%
AST	85.304.834	325,89	51,79	99,02%	0,98%
Res2Net	23.707.438	91,01	44,58	99,69%	0,36%
Conformer	15.925.122	61,27	45,34	99,82%	0,18%
RawNet2	7.004.418	26,90	42,33	97,16%	2,71%
AASIST	1.859.842	7,27	53,86	95,02%	4,89%
RawGAT-ST	241.024	1,02	33,37	93,60%	6,22%
WavLM Original	94.775.935	361,63	18,73	99,69%	0,36%
HuBERT Original	94.765.711	361,58	17,00	99,87%	0,18%

Nota sobre latência: mediana de 30 inferências de amostra única (duas execuções de aquecimento), medida no ambiente de execução de cada modelo — CPU para CCT, Conformer, SVM e Random Forest; GPU (RTX 3060) para os demais. Os valores não são, portanto, diretamente comparáveis entre dispositivos (Seção 5.1).

Tabela 10: Robustez a ruído AWGN, aplicada à forma de onda antes dos frontends, por modelo consolidado.

Modelo	Limp	30 dB	20 dB	10 dB
Random Forest	97,82%	95,69%	94,22%	92,36%
SVM	99,24%	97,96%	96,04%	93,78%
CCT	99,20%	98,84%	97,96%	94,98%
AST	99,02%	97,78%	96,44%	93,24%
Res2Net	99,69%	99,24%	99,02%	97,51%
Conformer	99,82%	99,64%	99,47%	97,96%
RawNet2	97,16%	95,47%	95,33%	92,98%
AASIST	95,02%	92,98%	91,51%	88,71%
RawGAT-ST	93,60%	93,29%	91,07%	83,96%
WavLM Original	99,69%	99,51%	99,56%	98,76%
HuBERT Original	99,87%	99,38%	98,89%	96,80%

5.3 Análise dos Resultados

Desempenho no Conjunto Limpo

O recorte consolidado da Tabela 8 e da Figura 2 mostra maior desempenho para HuBERT e Conformer, com 99,87% e 99,82% de acurácia no conjunto de teste limpo (ambos com EER de 0,18%). Em seguida aparecem Res2Net e WavLM (ambos 99,69%, EER de 0,36%), SVM (99,24%) e CCT (99,20%), indicando que redes neurais profundas e classificadores estatísticos otimizados extraem padrões discriminativos relevantes no cenário experimental controlado (QP1). Com 2.250 amostras de teste, uma acurácia de 99,87% corresponde a apenas 3 amostras mal classificadas, e a diferença entre HuBERT e Conformer equivale a uma única amostra. HuBERT e Conformer devem, portanto, ser tratados como **empate técnico**, e não como ordenamento estrito entre primeiro e segundo colocados. O teste exato de McNemar sobre as predições pareadas confirma essa leitura: dos 2.250 casos de teste, apenas 3 são acertados exclusivamente pelo Conformer e 4 exclusivamente pelo HuBERT ($p = 1,0$); o mesmo vale para os pares Res2Net–Conformer (4 contra 7 desacordos, $p = 0,55$) e SVM–CCT (18 contra 17 desacordos, $p \approx 1,0$). Já a diferença entre Conformer e AST é estatisticamente significativa (22 acertos exclusivos do Conformer contra 4 do AST, $p < 0,001$).

Esse ordenamento se alinha apenas em parte à hierarquia sugerida pela Tabela 2. Os *backbones* SSL WavLM e HuBERT (4ª geração), avaliados com *backbone* congelado, soma ponderada das 13 camadas ocultas e *pooling* média $\oplus$ desvio sobre janela de 4 s (Seção 3.1),

Tabela 11: Resumo descritivo das curvas de validação em execução única.

Modelo	Val. pico	Época	Val. final	Queda	Nota
Random Forest	–	CV+fit	–	–	não aplicável (modelo clássico)
SVM	–	CV+fit	–	–	não aplicável (modelo clássico)
CCT	99,64%	57	99,56%	0,09%	descritivo; 1 execução
AST	99,02%	53	98,93%	0,09%	descritivo; 1 execução
Res2Net	99,82%	47	99,78%	0,04%	descritivo; 1 execução
Conformer	99,87%	41	99,73%	0,13%	descritivo; 1 execução
RawNet2	97,51%	16	97,16%	0,36%	descritivo; 1 execução
AASIST	95,51%	34	93,42%	2,09%	descritivo; 1 execução
RawGAT-ST	94,09%	23	93,69%	0,40%	descritivo; 1 execução
WavLM Original	99,73%	97	99,47%	0,27%	descritivo; 1 execução
HuBERT Original	99,78%	82	99,47%	0,31%	descritivo; 1 execução

figuram no topo do recorte, consistentes com a literatura. Já AASIST e RawGAT-ST (3ª geração, grafos de atenção sobre áudio bruto) são os dois mais fracos do conjunto: AASIST registra EER de 4,89% e RawGAT-ST de 6,22% — bem acima dos valores reportados pelos artigos originais no ASVspoof 2019 LA: EER de 0,83% para o AASIST [32], 1,06% para o RawGAT-ST [40] e $\approx 1,1\%$ para o RawNet2 [31]. Essa distância não invalida os modelos: ela reflete diferenças de escopo entre o *benchmark* deste trabalho e os artigos originais. As implementações de AASIST/RawGAT-ST são variantes alinhadas aos artigos, não réplicas exatas, treinadas com janela de 1 s e *augmentation* dinâmico por época; e o *corpus* deste trabalho, em português e com XTTS como único gerador da fração Fake Voices, é distinto e mais restrito que o ASVspoof, que combina dezenas de sistemas de ataque. A hierarquia observada é, portanto, específica deste protocolo e não constitui refutação dos resultados publicados para essas arquiteturas.

A decomposição dos erros por tipo (Figura 9) revela padrões operacionalmente distintos. A maioria dos modelos erra de forma aproximadamente simétrica (Conformer: 2 falsos positivos contra 2 falsos negativos; AST: 12 contra 10; SVM: 8 contra 9), mas AASIST e RawGAT-ST concentram os erros em falsos positivos (AASIST: 75 amostras *bonafide* rejeitadas contra 37 *spoofs* aceitos; RawGAT-ST: 122 contra 22), o que penaliza a usabilidade pela rejeição indevida de usuários legítimos. Já o RawNet2 apresenta excesso de falsos negativos (51 *spoofs* aceitos contra 13 rejeições indevidas) — o erro crítico no cenário de biometria em *tandem*, em que aceitar um ataque custa mais do que rejeitar um usuário genuíno ($C_{fa} = 10 \gg C_{miss} = 1$; Seção 4.5).

Importância de Características no Vetor Tabular

A Figura 10 apresenta a importância de Gini das características do vetor tabular de 63 descritores (Seção 4.3), extraída diretamente do artefato treinado do *Random Forest* (`scripts/reporting/export_rf_feature_importance.py`). Agregando por família, os coeficientes cepstrais MFCC concentram 49,9% da importância total, seguidos por RASTA-PLP (35,2%) e pelas estatísticas temporais (14,9%). Individualmente, a média

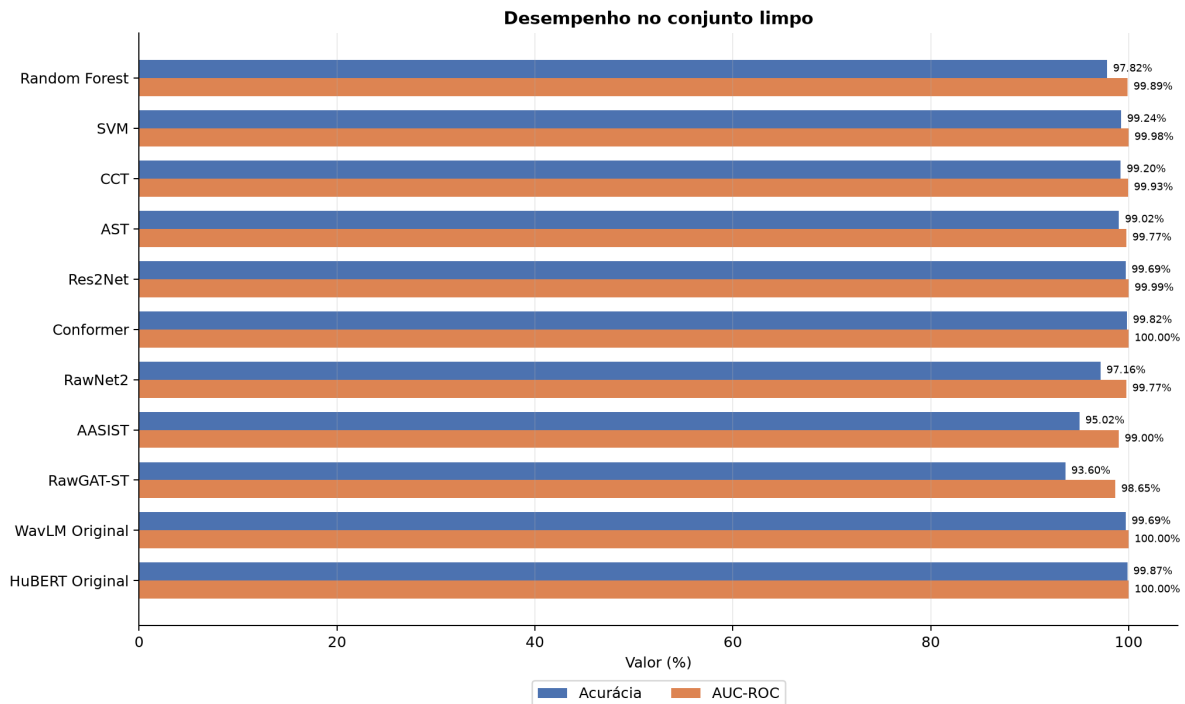


Figura 2: Acurácia e AUC-ROC no conjunto de teste limpo.

do MFCC3 é o descritor mais informativo (5,6%), e 11 das 15 características mais importantes são cepstrais.

Duas leituras decorrem desse resultado. Primeiro, a dominância cepstral confirma que a informação discriminativa entre fala genuína e sintética no *corpus* está majoritariamente na envoltória espectral — coerente com a literatura de *anti-spoofing* de primeira geração [27] — enquanto estatísticas de amplitude contribuem pouco. Segundo, a participação relevante do RASTA-PLP (35,2%) mostra que o classificador de fato utiliza os descritores projetados para robustez de canal; ainda assim, o *Random Forest* é um dos modelos menos robustos sob ruído (Tabela 10), reforçando que a fragilidade dos clássicos decorre da natureza agregada do vetor — médias e desvios globais por amostra descartam a estrutura tempo-frequência fina que as redes espectrais exploram — e não da ausência de características apropriadas.

Robustez a Ruído: o Achado Central

A Tabela 10 e a Figura 3 mostram degradação suave e monotônica em todo o recorte sob o protocolo *waveform-AWGN* (ruído aplicado à forma de onda antes de qualquer *frontend*, com exemplos ruidosos compatíveis também no treino). Os mais robustos a 10 dB são WavLM Original (98,8%), Conformer (98,0%), Res2Net (97,5%) e HuBERT Original (96,8%), seguidos por CCT (95,0%), SVM (93,8%), AST (93,2%), RawNet2 (93,0%) e Random Forest (92,4%); AASIST (88,7%) e RawGAT-ST (84,0%) fecham o conjunto — nenhum deles próximo do acaso (50%).

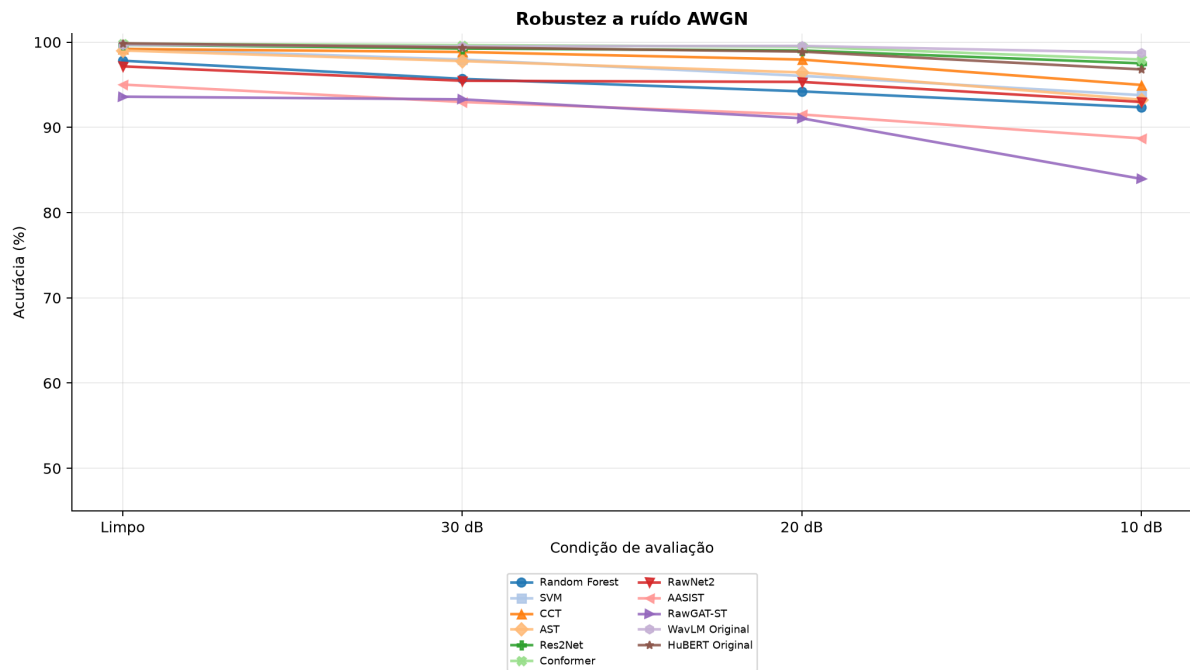


Figura 3: Robustez a AWGN na forma de onda (acurácia vs. SNR).

Dois achados emergem dessa leitura (QP2). Primeiro, o casamento entre o ruído de treino e o de teste importa mais do que a família arquitetural: com o protocolo *waveform*-AWGN, os classificadores clássicos — que em protocolos anteriores, com ruído aplicado apenas na avaliação, despencavam para 66–68% — passam a degradar suavemente (SVM: 99,24% → 93,8%; Random Forest: 97,82% → 92,4%), mesmo limitados ao vetor de 63 descritores agregados. Segundo, a fragilidade relativa se concentra nas redes de grafos sobre áudio bruto: RawGAT-ST perde 9,6 pontos percentuais entre o conjunto limpo e 10 dB (93,60% → 84,0%) e AASIST perde 6,3 (95,02% → 88,7%) — as maiores quedas do recorte, indicando que a janela curta de 1 s e a atenção em grafos sobre a forma de onda são mais sensíveis ao ruído aditivo do que as representações tempo-frequência densas de Conformer, Res2Net e AST ou os *embeddings* SSL pré-treinados em 94 mil horas de fala.

Ponto de Operação: AUC, EER e t -DCF*

Duas leituras complementares emergem das métricas independentes de limiar. Primeiro, a comparação entre AASIST e RawGAT-ST é consistente em todas as métricas: o AASIST tem EER menor (4,89% contra 6,22%), AUC-ROC maior (0,990 contra 0,986) e t -DCF* menor (0,1267 contra 0,1745), e o teste de McNemar pareado confirma a diferença (87 acertos exclusivos do AASIST contra 55 do RawGAT-ST, $p < 0,01$). As distribuições de pontuação, porém, têm formas distintas (Figura 11): a cabeça AM-Softmax do AASIST, por maximizar margem angular, satura as pontuações nos extremos do intervalo (98,0% dos *scores* abaixo de 0,05 ou acima de 0,95, contra 86,5% no RawGAT-ST). Essa

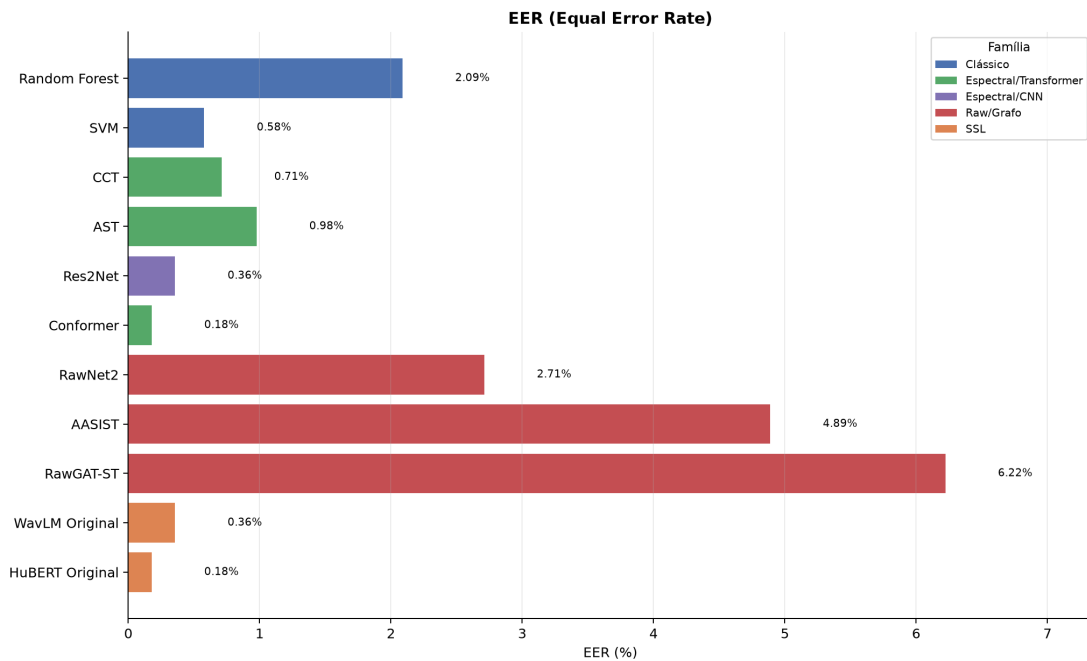


Figura 4: Taxa de Erro Igual (EER) por arquitetura.

saturação produz separação nítida no ponto de equilíbrio, mas deixa uma cauda de 3,2% dos *spoofs* com pontuação extrema no lado incorreto, visível na curva DET do AASIST (Figura 12) como um piso de $P_{miss} \approx 3\%$ pouco sensível ao limiar — um erro residual que recalibração de limiar não remove.

Segundo, o t -DCF* acompanha o ordenamento do EER, mas amplia a distância dos dois modelos de grafos: RawGAT-ST registra 0,1745 e AASIST 0,1267, contra valores entre 0,0027 (HuBERT Original) e 0,0098 nos demais — razões maiores que as respectivas razões de EER. Isso ocorre porque a parametrização adotada (ASVspoof 2019 LA, com $C_{fa} = 10 \gg C_{miss} = 1$) pondera fortemente a região de baixa taxa de falso-alarme, onde a cauda de *spoofs* com pontuação saturada no lado incorreto (discutida acima) mantém P_{miss} elevado. Para o cenário de verificação de locutor em *tandem* que a métrica modela (Seção 1.1), AASIST e RawGAT-ST são, portanto, menos adequados do que o EER isolado sugeriria — ilustrando por que o protocolo reporta as três métricas em conjunto.

Vazamento de Domínio e Separabilidade

Os valores de AUC-ROC muito próximos de 1,000 (Res2Net, Conformer) e os picos de acurácia de validação de até 100% (Conformer, época 17 — Tabela 11) devem ser lidos à luz da análise de vazamento de domínio da Seção 4.4.3. Separabilidade quase perfeita é compatível tanto com um detector genuinamente eficaz quanto com um classificador que explora correlações espúrias entre fonte, canal e rótulo. A robustez sob ruído não discrimina claramente os modelos de melhor desempenho limpo dos demais — Res2Net, Conformer, AST, RawNet2, AASIST, RawGAT-ST, CCT, HuBERT Original e WavLM Original preservam todos entre aproximadamente 76% e 98% de acurácia a 10 dB — o

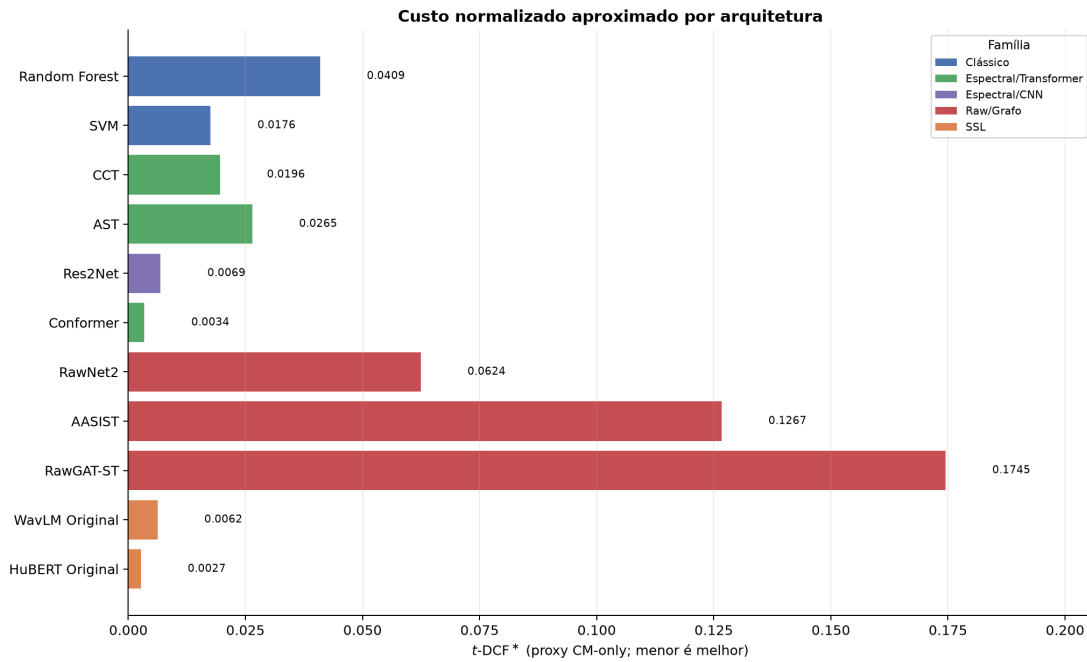


Figura 5: t -DCF* por arquitetura.

que remove um possível argumento indireto de robustez como evidência de generalização genuína para Res2Net/Conformer, mas também não confirma nem afasta a hipótese de vazamento: a separabilidade quase perfeita desses dois modelos permanece compatível com ambas as leituras. A confirmação exige validação *cross-dataset* com fontes disjuntas (Seção 6.3).

Eficiência Computacional e Trade-offs

A Tabela 9 e as Figuras 6 e 7 mostram que desempenho e custo computacional não são monotonicamente relacionados (QP3). Uma ressalva metodológica antecede a leitura: as latências foram medidas no ambiente de execução de cada modelo (CPU para CCT, Conformer, SVM e Random Forest; GPU para os demais — Seção 5.1), de modo que os valores absolutos não formam um *ranking* único entre famílias; os 18,73 ms do WavLM Original e os 17,00 ms do HuBERT Original, por exemplo, refletem inferência em GPU e não são comparáveis aos 45,34 ms do Conformer em CPU. Feita a ressalva, RawGAT-ST é o modelo mais compacto do conjunto (241.024 parâmetros, 1,02 MB) e, apesar disso, mantém acurácia limpa de 93,60% e robustez de 84,0% a 10 dB; SVM alia latência mínima em CPU (0,28 ms) a acurácia competitiva em condição limpa (99,24%) e robustez de 93,8% a 10 dB, tornando-se uma opção atrativa para cenários com restrição severa de recursos. Conformer, um dos mais robustos sob ruído, ocupa posição intermediária de custo (15,9 milhões de parâmetros, 61,27 MB, 45,34 ms em CPU); os *backbones* SSL, líderes em acurácia e robustez, são os mais pesados do conjunto ($\approx 94,8$ milhões de parâmetros, ≈ 362 MB), seguidos pelo AST (85,3 milhões, 325,89 MB). Não há, portanto, um único modelo dominante em todas as dimensões: a escolha depende de qual restrição — latência,

memória ou robustez — é prioritária na aplicação-alvo.

Estabilidade de Treinamento

A Tabela 11 e a Figura 8 mostram que a maioria dos modelos neurais converge de forma estável, com quedas entre o pico de validação e o valor final inferiores a 0,5%. A principal exceção é o AASIST (queda de 2,09%, pico de 95,51% na época 34), com flutuação moderada de validação ao longo do treino. Os picos de validação ocorrem em épocas variadas — Conformer na época 37, RawGAT-ST na 42, CCT na 57, Res2Net na 72, WavLM e HuBERT Original na 81 — sem convergência precoce sob o protocolo com *augmentation* de ruído no treino. O Conformer atinge 99,87% de acurácia de validação, sinal de que a tarefa de validação é, nesse ponto, quase trivialmente separável. Esse resultado reforça a leitura cautelosa sobre vazamento de domínio discutida acima, em vez de ser apenas um indicador positivo de convergência.

Ressalva sobre Execução Única

Todos os resultados reportados correspondem a uma única execução por modelo, sem repetição sobre múltiplas sementes aleatórias de inicialização e particionamento. Diferenças pequenas entre modelos próximos, como HuBERT e Conformer (99,87% vs. 99,82% de acurácia), não devem ser interpretadas como estatisticamente significativas. Uma régua útil é o intervalo de confiança de Wilson a 95% para a acurácia, que quantifica apenas a incerteza amostral do conjunto de teste ($n = 2.250$): ele varia de $\pm 0,17$ ponto percentual no topo (99,87% $\rightarrow$ [99,61%; 99,95%]) a $\pm 1,0$ ponto no modelo de menor acurácia (93,60% $\rightarrow$ [92,51%; 94,54%]). Sob essa régua — e sob o teste exato de McNemar pareado, que resulta em $p \approx 1,0$ para os pares HuBERT–Conformer e SVM–CCT —, o empate SVM–CCT no conjunto limpo (99,24% vs. 99,20%) é indistinguível, enquanto as diferenças de robustez a 10 dB discutidas acima — de até 14 pontos percentuais entre os extremos — excedem em muito a incerteza amostral. Note-se que esse intervalo não captura a variância de treinamento (inicialização, ordem dos lotes), que só múltiplas execuções revelariam. A magnitude dos efeitos discutidos acima (robustez a ruído e diferença entre famílias de modelos) é suficiente para uma interpretação qualitativa, mas comparações finas entre os modelos de melhor colocação exigiriam múltiplas execuções com relato de média e desvio-padrão.

A leitura das métricas deve ser feita, portanto, com parcimônia: os resultados caracterizam desempenho em distribuição fechada e dependem do *corpus*, das fontes de geração e do protocolo de particionamento utilizado. A eficiência computacional, a estabilidade e, sobretudo, a robustez a ruído são parte essencial da interpretação, não apenas complementos gráficos ao desempenho no conjunto limpo.

6 Conclusões

Este trabalho desenvolveu e avaliou um *pipeline* modular e de código aberto para detecção de áudio sintético em português, submetendo 11 modelos heterogêneos — de classificadores estatísticos clássicos a redes espectro-temporais e *backbones* auto-supervisionados — a um protocolo comum de treinamento, inferência e *benchmark*. Nesse sentido, os objetivos específicos e as questões de pesquisa delineados na Seção 1 foram atendidos: consolidou-se uma base balanceada com particionamento reprodutível, implementou-se o pré-processamento acústico e a extração de características, e os modelos foram comparados sob métricas de acurácia, F1-score, AUC-ROC, EER, *t*-DCF, latência, tamanho de artefato e robustez a ruído.

No conjunto de teste limpo, os *backbones* auto-supervisionados e as arquiteturas espectro-temporais densas lideraram: HuBERT Original e Conformer alcançaram 99,87% e 99,82% de acurácia (ambos com EER de 0,18% e AUC-ROC $\approx 1,000$), seguidos por Res2Net e WavLM Original (ambos 99,69%), SVM (99,24%) e CCT (99,20%). Esse ordenamento (QP1) confirma que representações tempo-frequência ricas, associadas a mecanismos de atenção ou convoluções multi-escala — e *embeddings* SSL pré-treinados em larga escala, lidos por soma ponderada de camadas — capturam de modo eficaz os artefatos de síntese presentes no *corpus* avaliado.

O contraste entre desempenho limpo e robustez sob ruído (QP2) mostra degradação suave em todo o recorte sob o protocolo *waveform*-AWGN, com o ruído de treino casado ao de teste: WavLM, Conformer, Res2Net e HuBERT preservaram acima de 96% a 10 dB de SNR, os classificadores clássicos ficaram em 92–94%, e as redes de grafos sobre áudio bruto foram as mais sensíveis — AASIST a 88,7% e RawGAT-ST a 84,0% (Seção 5.3). Essa leitura evidencia que a robustez depende menos da família arquitetural e mais do casamento entre as condições de treino e de avaliação: com exemplos ruidosos compatíveis no treino, até o vetor de descritores agregados de SVM/*Random Forest* degrada de forma controlada, enquanto a janela curta e a atenção em grafos de AASIST/RawGAT-ST concentram as maiores quedas do conjunto.

A leitura conjunta desses resultados com a análise de vazamento de domínio (Seção 4.4.3) recomenda cautela interpretativa. Os valores de AUC-ROC próximos da unidade, os picos de acurácia de validação próximos de 100% e a alta separabilidade geral são compatíveis com a hipótese de que parte do sinal aprendido decorre de correlações espúrias entre fonte, canal e rótulo, e não exclusivamente das assinaturas do processo de síntese. Assim, as métricas reportadas devem ser entendidas como limite superior de desempenho *in-domain*, não como estimativa de generalização para cenários reais.

Por fim, os *backbones* auto-supervisionados WavLM e HuBERT, mesmo congelados e com apenas a cabeça de classificação treinada, alcançaram o topo do recorte (99,69% e 99,87% no conjunto limpo; 98,8% e 96,8% a 10 dB) — resultado obtido com a leitura

por soma ponderada das 13 camadas ocultas, *pooling* média $\oplus$ desvio e janela de 4s, em linha com o protocolo SUPERB. Esse comportamento é consistente com a literatura sobre representações SSL para anti-*spoofing* e delimita a condição sob a qual os modelos foram avaliados: o ajuste fino integral do *backbone* permanece como caminho para cenários mais adversos, ao custo de duas ordens de grandeza a mais de parâmetros treináveis.

Sob a ótica de implantação (QP3), nenhum modelo domina simultaneamente acurácia, robustez, latência e memória (Seção 5.3): o Conformer oferece o melhor compromisso em CPU (acurácia e robustez de topo, custo intermediário de 61 MB e 45 ms), os *backbones* SSL trocam ≈ 362 MB de artefato e inferência em GPU pelo melhor desempenho conjunto limpo+ruído, e o SVM alia latência mínima em CPU (0,3 ms) a acurácia e robustez competitivas, tornando-se a opção natural sob restrição severa de recursos — lembrando que as latências foram medidas em dispositivos distintos por família e valem como ordens de grandeza (Seção 5.3). A escolha do detector deve, portanto, partir das restrições da aplicação-alvo, não de um ranking único.

6.1 Principais Contribuições

- Um *pipeline* reproduzível e de código aberto que padroniza pré-processamento, treinamento, inferência e geração automática de relatórios de *benchmark* para detecção de áudio sintético em português.
- Uma comparação empírica de 11 arquiteturas heterogêneas sob protocolo único, contemplando eficiência computacional e robustez a ruído AWGN, em *corpus* de fala em português, com predominância da variante brasileira.
- Uma análise crítica de vazamento de domínio que documenta os vieses de fonte e canal do protocolo e relativiza as métricas obtidas, contribuindo para a interpretação responsável de resultados em detecção de *deepfakes*.

6.2 Limitações

As conclusões restringem-se a um cenário de distribuição fechada. As principais limitações são: (i) a ausência de disjunção estrita de locutores em parte do *corpus*; (ii) o pareamento assimétrico entre fontes reais e sintéticas, que induz vazamento de domínio; (iii) a avaliação dos modelos SSL apenas com *backbone* congelado; (iv) o uso de um *t*-DCF na forma de *proxy* interno, sem acoplamento a um sistema ASV oficial; (v) resultados de execução única por modelo, sem repetição sobre múltiplas sementes aleatórias, o que impede estimar variância entre execuções e limita a significância estatística de diferenças pequenas entre os modelos de melhor colocação; e (vi) o limiar de decisão das métricas dependentes de limiar é calibrado sob ruído para os dois modelos SSL e fixo em 0,5 para os demais — assimetria pequena ($|\theta - 0,5| < 0,02$) e declarada na Seção 4.5, mas que a

uniformização da calibração para todos os modelos eliminaria; e (vii) as latências de inferência foram medidas no ambiente de execução de cada modelo (CPU ou GPU, conforme a família), o que permite leitura por ordem de grandeza, mas não um *ranking* único de latência entre todos os modelos.

6.3 Trabalhos Futuros

Como continuidade, recomenda-se ampliar a validação para partições estritamente disjuntas por locutor e por base de origem, incluir ensaios com geradores não vistos (*cross-dataset*) e consolidar os experimentos com modelos auto-supervisionados ajustados integralmente em GPU, particularmente WavLM e HuBERT. No protocolo de avaliação, duas extensões diretas são (a) estender a calibração de limiar sob ruído — hoje aplicada apenas aos modelos SSL — a todas as arquiteturas, uniformizando o ponto de operação (limitação (vi)); e (b) repetir o *benchmark* sobre múltiplas sementes aleatórias com relato de média e desvio-padrão, quantificando a variância de treinamento que o intervalo de confiança amostral não captura. Destaca-se ainda a pertinência de adotar o *t*-DCF padronizado acoplado a um sistema ASV de referência, incorporar métricas de calibração e explicabilidade local (SHAP, GradCAM), conduzir testes de robustez contra compressão, reverberação e ruído real de canal, re-medir a latência de todos os modelos em um único dispositivo de referência (limitação (vii)) e controlar explicitamente a proporção de fontes por partição, aproximando o protocolo experimental de cenários operacionais de biometria de voz.

Referências

- [1] JIA, Y. et al. Transfer learning from speaker verification to multispeaker text-to-speech synthesis. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 31, 2018.
- [2] DUDLEY, H. The vocoder. **Bell Labs Record**, v. 18, n. 4, p. 122–126, 1939.
- [3] KLATT, D. H. Review of text-to-speech conversion for English. **The Journal of the Acoustical Society of America**, v. 82, n. 3, p. 737–793, 1987.
- [4] HUNT, A. J.; BLACK, A. W. Unit selection in a concatenative speech synthesis system using a large speech database. In: **IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)**, 1996.
- [5] VAN DEN OORD, A. et al. WaveNet: A generative model for raw audio. **arXiv preprint** arXiv:1609.03499, 2016.
- [6] WANG, Y. et al. Tacotron: Towards end-to-end speech synthesis. In: **Proc. Interspeech 2017**, 2017.
- [7] WANG, C. et al. VALL-E: Neural Codec Language Models are Zero-Shot Text-to-Speech Synthesizers. **arXiv preprint** arXiv:2301.02111, 2023. Disponível em: <https://arxiv.org/abs/2301.02111>. Acesso em: 15 jun. 2025.
- [8] MAI, K. T. et al. Warning: humans cannot reliably detect speech deepfakes. **PLOS ONE**, v. 18, n. 8, e0285333, 2023.
- [9] HEIDARI, A. et al. Deepfake detection using deep learning methods: A systematic and comprehensive review. **WIREs Data Mining and Knowledge Discovery**, v. 14, n. 2, e1520, 2024.
- [10] SHEN, J. et al. Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 4779–4783, 2018.
- [11] BIRD, J. J.; LOTFI, A. Real-time Detection of AI-Generated Speech for DeepFake Voice Conversion. **arXiv preprint** arXiv:2308.12734, 2023. Disponível em: <https://arxiv.org/abs/2308.12734>. Acesso em: 15 jun. 2025.
- [12] PINDROP. Deepfake Trends to Look Out for in 2025. 2025. Disponível em: <https://www.pindrop.com/article/deepfake-trends/>. Acesso em: 15 jun. 2025.

- [13] FARRUGIA, B. Brazil's electoral deepfake law tested as AI-generated content targeted local elections. **Digital Forensic Research Lab (Atlantic Council)**, 26 nov. 2024. Disponível em: <https://dfrlab.org/2024/11/26/brazil-election-ai-deepfakes/>. Acesso em: 15 jun. 2025.
- [14] FARRUGIA, B. The challenges of identifying deepfakes ahead of the 2024 Brazil election. **Digital Forensic Research Lab (Atlantic Council)**, 2 out. 2024. Disponível em: <https://dfrlab.org/2024/10/02/brazil-election-ai-research/>. Acesso em: 15 jun. 2025.
- [15] TODISCO, M. et al. ASVspoof 2019: Future Horizons in Spoofed and Fake Speech Detection. In: **Proc. Interspeech 2019**, p. 1008–1012, 2019.
- [16] YAMAGISHI, J. et al. ASVspoof 2021: Accelerating Progress in Spoofed and Deepfake Speech Detection. In: **Proc. ASVspoof 2021 Workshop**, p. 47–54, 2021.
- [17] WANG, X. et al. ASVspoof 5: Crowdsourced Speech Data, Deepfakes, and Adversarial Attacks at Scale. **arXiv preprint** arXiv:2408.08739, 2024. Disponível em: <https://arxiv.org/abs/2408.08739>. Acesso em: 15 jun. 2025.
- [18] YI, J. et al. ADD 2022: The First Audio Deep Synthesis Detection Challenge. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 9216–9220, 2022.
- [19] REN, Y. et al. FastSpeech: Fast, Robust and Controllable Text to Speech. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 32, 2019.
- [20] KIM, J.; KONG, J.; SON, J. Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech. In: **International Conference on Machine Learning (ICML)**, p. 5530–5540, 2021.
- [21] CASANOVA, E. et al. YourTTS: Towards Zero-Shot Multi-Speaker TTS and Zero-Shot Voice Conversion for everyone. In: **International Conference on Machine Learning (ICML)**, p. 2709–2720, 2022.
- [22] UNFAKE. Fake Voices dataset card. **Hugging Face Datasets**, 2026. Disponível em: https://huggingface.co/datasets/unfake/fake_voices. Acesso em: 27 jun. 2026.
- [23] KAMEOKA, H. et al. StarGAN-VC: Non-parallel many-to-many voice conversion using star generative adversarial networks. In: **IEEE Spoken Language Technology Workshop (SLT)**, p. 266–273, 2018.
- [24] CORTES, C.; VAPNIK, V. Support-vector networks. **Machine Learning**, v. 20, p. 273–297, 1995.

- [25] HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.
- [26] WU, Z. et al. ASVspooF 2015: the First Automatic Speaker Verification Spoofing and Countermeasures Challenge. In: **Proc. Interspeech 2015**, p. 2037–2041, 2015.
- [27] SAHIDULLAH, M. et al. A Comparison of Features for Synthetic Speech Detection. In: **Proc. Interspeech 2015**, p. 2087–2091, 2015.
- [28] TODISCO, M.; DELGADO, H.; EVANS, N. A New Feature for Automatic Speaker Verification Anti-Spoofing: Constant Q Cepstral Coefficients. In: **Proc. Odyssey 2016 — The Speaker and Language Recognition Workshop**, p. 283–290, 2016.
- [29] LAVRENTYEVA, G. et al. Audio Replay Attack Detection with Deep Learning Frameworks. In: **Proc. Interspeech 2017**, p. 82–86, 2017.
- [30] GAO, S. et al. Res2Net: A New Multi-Scale Backbone Architecture. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 43, n. 2, p. 652–662, 2021.
- [31] TAK, H. et al. End-to-End Anti-Spoofing with RawNet2. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 6369–6373, 2021.
- [32] JUNG, J. W. et al. AASIST: Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 6367–6371, 2022.
- [33] CHEN, S. et al. WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing. **IEEE Journal of Selected Topics in Signal Processing**, v. 16, n. 6, p. 1505–1518, 2022.
- [34] HSU, W.-N. et al. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. **IEEE/ACM Transactions on Audio, Speech, and Language Processing**, v. 29, p. 3451–3460, 2021.
- [35] BAEVSKI, A. et al. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. **Advances in Neural Information Processing Systems (NeurIPS)**, v. 33, p. 12449–12460, 2020.
- [36] GONG, Y.; CHUNG, Y.; GLASS, J. AST: Audio Spectrogram Transformer. In: **Proc. Interspeech 2021**, p. 571–575, 2021.

- [37] VASWANI, A. et al. Attention Is All You Need. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 30, 2017.
- [38] DOSOVITSKIY, A. et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In: **International Conference on Learning Representations (ICLR)**, 2021.
- [39] AKCIT-DEEPPFAKE. BRSpeech-DF dataset card. **Hugging Face Datasets**, 2026. Disponível em: <https://huggingface.co/datasets/AKCIT-Deepfake/BRSpeech-DF>. Acesso em: 27 jun. 2026.
- [40] TAK, H. et al. End-to-End Spectro-Temporal Graph Attention Networks for Speaker Verification Anti-Spoofing and Speech Deepfake Detection. In: **Proc. ASVspoof 2021 Workshop**, p. 1–8, 2021.
- [41] GULATI, A. et al. Conformer: Convolution-augmented Transformer for Speech Recognition. In: **Proc. Interspeech 2020**, p. 5036–5040, 2020.
- [42] HASSANI, A. et al. Escaping the Big Data Paradigm with Compact Transformers. **arXiv preprint** arXiv:2104.05704, 2021. Disponível em: <https://arxiv.org/abs/2104.05704>. Acesso em: 28 jun. 2026.
- [43] MÜLLER, N. M. et al. Does Audio Deepfake Detection Generalize? In: **Proc. Interspeech 2022**, p. 2783–2787, 2022.
- [44] BREIMAN, L. Random Forests. **Machine Learning**, v. 45, p. 5–32, 2001.
- [45] HERMANSKY, H.; MORGAN, N. RASTA Processing of Speech. **IEEE Transactions on Speech and Audio Processing**, v. 2, n. 4, p. 578–589, 1994.
- [46] PRATAP, V. et al. MLS: A Large-Scale Multilingual Dataset for Speech Research. **Proc. Interspeech 2020**, p. 2757–2761, 2020. Disponível em: <https://www.openslr.org/94/>. Acesso em: 28 jun. 2026.
- [47] CASANOVA, E. et al. TTS-Portuguese Corpus: a corpus for speech synthesis in Brazilian Portuguese. **Language Resources and Evaluation**, v. 56, p. 1043–1055, 2022. Disponível em: <https://github.com/Edresson/TTS-Portuguese-Corpus>. Acesso em: 28 jun. 2026.
- [48] KINNUNEN, T. et al. t-DCF: a Detection Cost Function for the Tandem Assessment of Spoofing Countermeasures and Automatic Speaker Verification. In: **Proc. Odyssey 2018 — The Speaker and Language Recognition Workshop**, p. 312–319, 2018.

-
- [49] LUNDBERG, S. M.; LEE, S.-I. A Unified Approach to Interpreting Model Predictions. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 30, p. 4765–4774, 2017.
- [50] KINGMA, D. P.; BA, J. Adam: A Method for Stochastic Optimization. In: **International Conference on Learning Representations (ICLR)**, 2015.

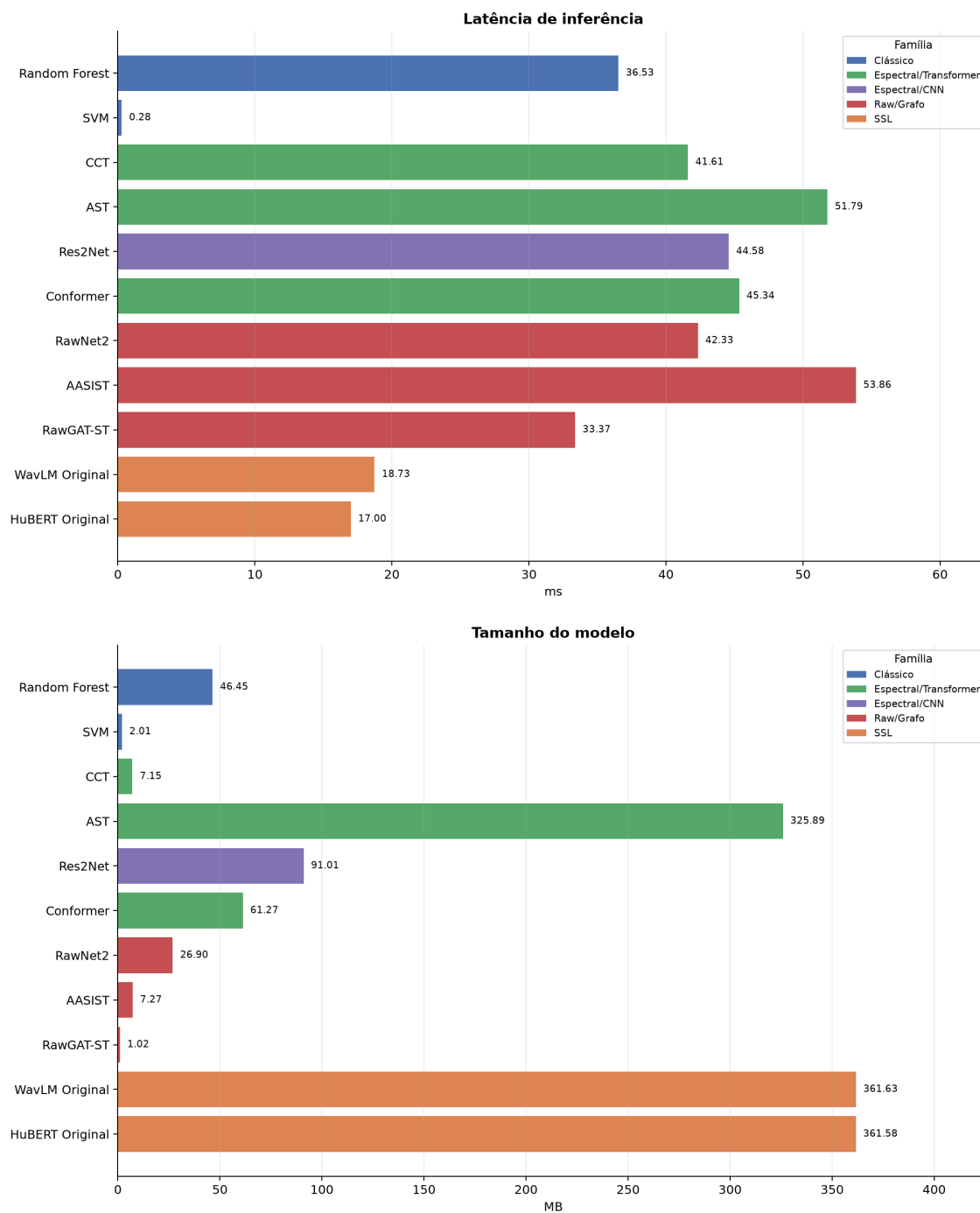


Figura 6: Latência e tamanho dos modelos.



Figura 7: Trade-off entre acurácia, latência e tamanho dos artefatos.

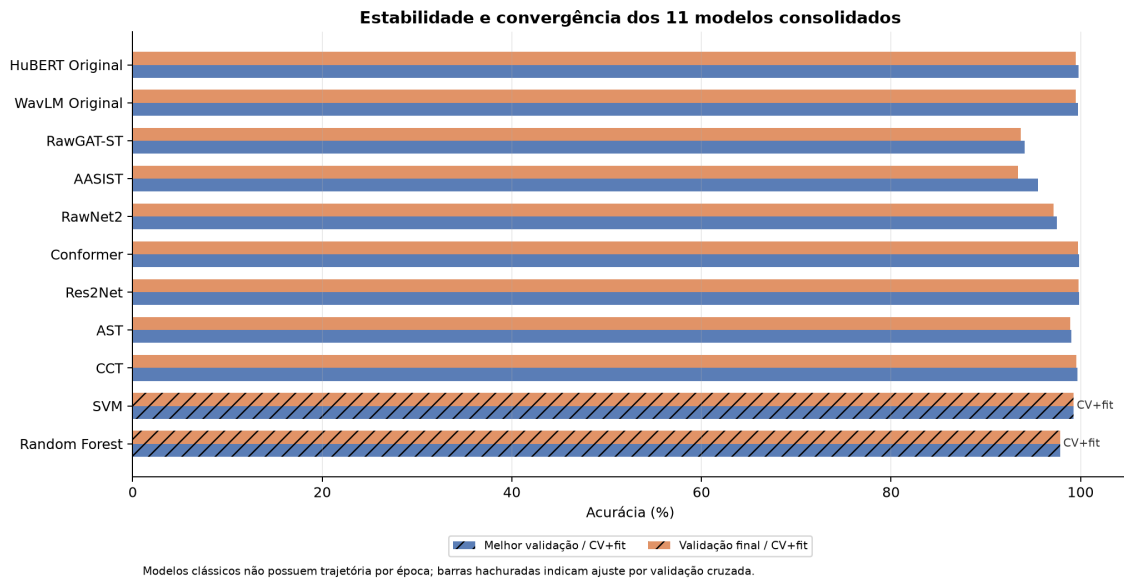


Figura 8: Resumo descritivo das curvas de validação em execução única; a diferença pico-final não demonstra, isoladamente, estabilidade de otimização. Modelos clássicos não possuem trajetória por época.

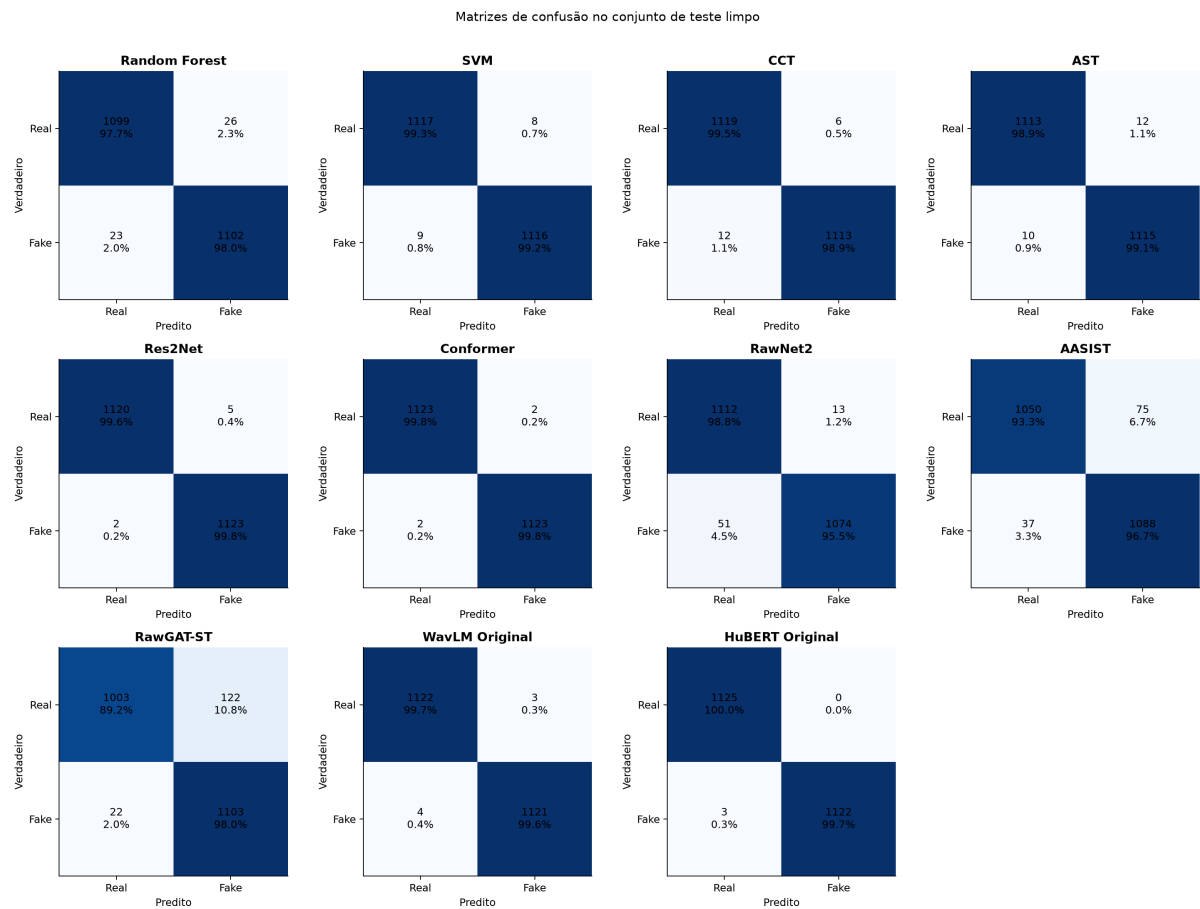


Figura 9: Matrizes de confusão dos onze modelos avaliados no artigo.

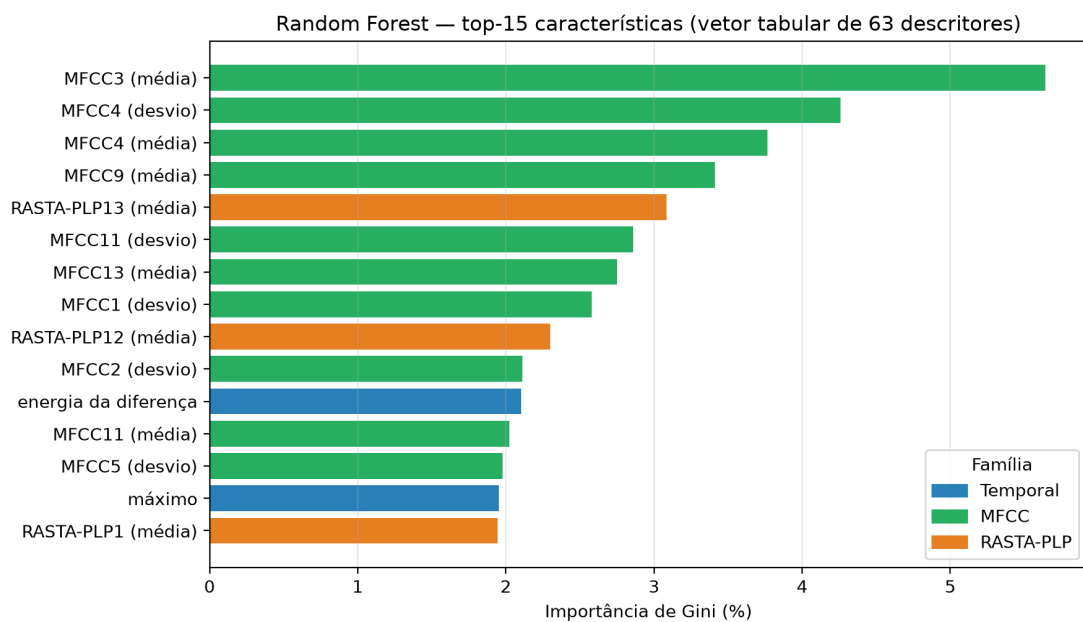


Figura 10: Importância de Gini das 15 características mais relevantes do *Random Forest*, coloridas por família (temporal, MFCC, RASTA-PLP). Agregadas, as famílias respondem por 14,9%, 49,9% e 35,2% da importância total, respectivamente.

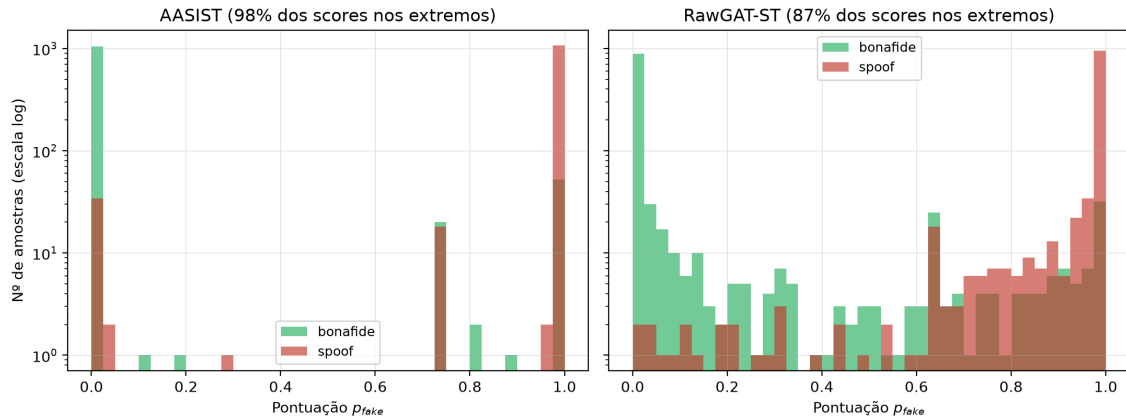


Figura 11: Distribuições da pontuação p_{fake} por classe verdadeira para AASIST e RawGAT-ST no conjunto de teste limpo, em escala logarítmica (geradas por `scripts/reporting/export_tcc_extra_figures.py`). A cabeça AM-Softmax do AASIST concentra 98,0% dos *scores* nos extremos do intervalo.

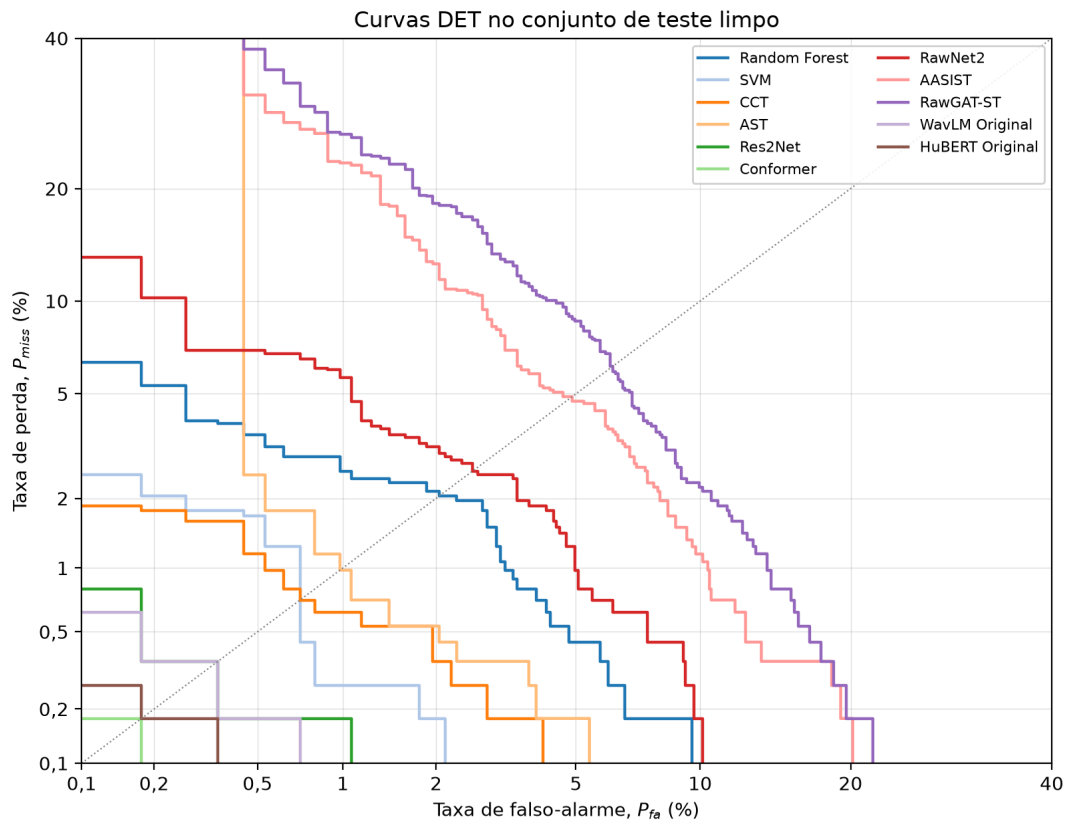


Figura 12: Curvas DET (*Detection Error Tradeoff*) dos 11 modelos no conjunto de teste limpo, em eixos probit (geradas por `scripts/reporting/export_tcc_extra_figures.py`). O piso de $P_{miss} \approx 3\%$ do AASIST evidencia a cauda de *spoofs* com pontuação saturada no extremo incorreto.

A Configurações Detalhadas do Pipeline

A.1 Hiperparâmetros Consolidados

O preset final do *benchmark* utiliza teto de 100 épocas para as redes espectrais e RawNet2 e de 120 épocas para AASIST, RawGAT-ST e para as cabeças de classificação SSL, com parada antecipada por perda de validação (exceto RawNet2, que treina o orçamento integral), batch adaptado ao consumo de VRAM, precisão mista quando segura e salvamento do melhor *checkpoint* com restauração dos melhores pesos. O preset contempla WavLM Original e HuBERT Original como modelos SSL no escopo principal do trabalho. Os modelos clássicos utilizam busca de hiperparâmetros com validação cruzada antes do ajuste final. A Tabela 13 resume esses parâmetros por família de arquitetura, e a Tabela 12 registra o ambiente de software usado nas execuções.

Tabela 12: Ambiente de software e parâmetros de reprodução das execuções.

Item	Valor
Python	3.11.15
TensorFlow/Keras	2.21.0
PyTorch	2.5.1 (CUDA 12.4)
Execução	Docker (WSL2/Linux), imagem <code>xfakesong/benchmark:nvidia</code>
GPU	NVIDIA GeForce RTX 3060 (CC 8.6, precisão mista FP16)
Semente aleatória	42 (particionamento, inicialização e AWGN)
Dataset	<code>app/datasets/benchmark_audio_raw_balanced_15k.npz</code>

Tabela 13: Hiperparâmetros efetivos por modelo, conforme registrados nos artefatos de cada execução (`metrics.json`). ES = paciência da parada antecipada, em épocas; Aug. = *augmentation* de ruído no treino. Otimizadores Adam/AdamW conforme [50].

Modelo	LR	Batch	Otimizador	Teto (épocas)	ES	Aug.
CCT	10^{-3}	32	AdamW	100	18	sim
AST	2×10^{-5}	8	AdamW	100	20	sim
Res2Net	2×10^{-3}	32	AdamW	100	15	sim
Conformer	10^{-3}	32	AdamW	100	22	sim
RawNet2	10^{-4}	16	Adam	100	—	sim
AASIST	3×10^{-4}	16	AdamW	120	25	sim
RawGAT-ST	5×10^{-5}	16	AdamW	120	25	sim
WavLM Original	10^{-3} (cabeça)	32	AdamW ($wd = 10^{-4}$)	120	15	sim
HuBERT Original	10^{-3} (cabeça)	32	AdamW ($wd = 10^{-4}$)	120	15	sim
SVM	GridSearchCV (5 dobras): <i>kernel</i> $\in$ {linear, RBF, polinomial}; <i>C</i> $\in$ {0,1; 1; 10; 100}; $\gamma \in$ {scale, auto, 10^{-3} , 10^{-2} , 10^{-1} , 1}; reajuste final no treino completo					
Random Forest	GridSearchCV (5 dobras): árvores $\in$ {200,300}; profundidade máx. $\in$ {10,15,20}; <i>min. split/leaf</i> $\in$ {5,10,20}/{2,4,8}; <i>max. features</i> $\in$ {sqrt, log2}					

A.2 Organização dos Artefatos

Cada arquitetura possui diretório próprio em **results/**, contendo modelo treinado, configuração, histórico de treinamento, métricas em JSON/CSV, matriz de confusão, curvas ROC/PR, gráficos de acurácia e perda, resultados de robustez e relatório Markdown. Os gráficos consolidados utilizados no TCC ficam em **figures/**.

B Implementação de Referência

A implementação avaliada neste trabalho corresponde ao *pipeline* real do repositório, e não a um exemplo isolado de código. A Tabela 14 registra os módulos efetivamente usados para treinamento, inferência, cálculo de métricas e geração dos relatórios.

Tabela 14: Módulos reais do *pipeline* referenciados pelo *benchmark*.

Etapa	Módulo	Responsabilidade
Orquestração	benchmarks/runner.py	Executa preparação, treino, inferência, métricas e persistência
Entrada operacional	scripts/benchmark/run_benchmark.py	Rodadas individuais ou múltiplas via linha de comando
Treinamento	app/domain/services/training_service.py	Cria o modelo, aplica callbacks, treina e salva arquivos gerados
Fábrica de arquiteturas	app/domain/models/architectures/factory.py	Normaliza aliases e instancia as arquiteturas avaliadas
Carregamento de modelos	app/domain/services/detection/model_loader.py	Recarrega modelos Keras/sklearn e contratos de entrada
Predição	app/domain/services/detection/predictor.py	Executa inferência padronizada e aplicação de limiar
Serviço de detecção	app/domain/services/detection_service.py	Fachada usada pela API/Gradio para inferência
Métricas	app/domain/models/training/metrics.py	Calcula métricas, curvas, EER e matriz de confusão
Relatórios	benchmarks/report.py	Consolida tabelas, figuras e Markdown para o TCC

Neste contexto, o *pipeline* de detecção de áudio sintético descrito neste trabalho corresponde à implementação de referência desenvolvida pelo autor como parte deste Trabalho de Conclusão de Curso.

C Comandos de Execução e Reprodutibilidade

Os comandos a seguir sintetizam o fluxo operacional usado para repetir o *pipeline*, executar modelos isolados e gerar os arquivos do TCC.

- **Preparar pastas locais:**

```
python main.py -bootstrap-dirs
```

- **Iniciar a interface Gradio:**

```
python main.py -gradio
```

- **Preservar o dataset original consolidado:**

```
dir app\datasets\benchmark_audio_raw_balanced_15k.npz
```

- **Executar o *pipeline* completo do TCC com o NPZ original:**

```
python scripts/benchmark/run_tcc_pipeline.py
-full-benchmark -epochs 100 -device-profile gpu
-npz app/datasets/benchmark_audio_raw_balanced_15k.npz
```

- Executar benchmark completo com o NPZ consolidado:

```
python scripts/benchmark/run_benchmark.py -full -epochs 100
-device-profile gpu
-dataset app/datasets/benchmark_audio_raw_balanced_15k.npz
```

WavLM Original e HuBERT Original usam um runner PyTorch/transformers dedicado, com backbone SSL real:

```
python scripts/benchmark/run_wavlm_original_benchmark.py
-architecture wavlm
-dataset app/datasets/benchmark_audio_raw_balanced_15k.npz
python scripts/benchmark/run_wavlm_original_benchmark.py
-architecture hubert
-dataset app/datasets/benchmark_audio_raw_balanced_15k.npz
```

- Executar benchmark de um modelo individual:

```
python scripts/benchmark/run_benchmark.py -model Conformer -epochs 100
-device-profile gpu
-dataset app/datasets/benchmark_audio_raw_balanced_15k.npz
```

- Executar testes automatizados:

```
pytest tests/
```

- Compilar o TCC:

```
pdflatex main.tex
```